



FlagEmbedding



One-Stop Retrieval Toolkit For Search and RAG

FlagEmbedding (BAAI)

FlagEmbedding

01 overview

FlagEmbedding ☐☐☐☐

--	--	--	--	--	--

FlagEmbedding (BGE)

Diagram 0

Mermaid Diagram

□□□□

項目	詳細	備考
1. 項目	詳細内容	備考
2. 項目	詳細内容	備考
3. 項目	詳細内容	備考
4. 項目	詳細内容	備考
5. 項目	詳細内容	備考
6. 項目	詳細内容	備考

1. Introduction

1.1 FlagEmbedding

FlagEmbedding [] Embedding [] Reranker [] BAAI []

1.2 快速上手

- 快速上手BAAI
- 快速上手BGE (BAAI General Embedding)
- 快速上手
- 快速上手
- 快速上手Encoder-only 与 Decoder-only
- 快速上手
- 快速上手

1.3 快速上手

FlagEmbedding 快速上手

- 快速上手
- 快速上手
- 快速上手
- 快速上手

2. 快速上手

2.1 快速上手

FlagEmbedding 快速上手

1. 快速上手**abc**/快速上手
2. 快速上手**inference**/快速上手
3. 快速上手**finetune**/快速上手
4. 快速上手**evaluation**/快速上手

2.2 快速上手

快速上手ABC快速上手

- [AbsEmbedder](#)快速上手 encode encode_queries encode_corpus 快速上手
- [AbsReranker](#)快速上手
- 快速上手 AbsModeling AbsDataset AbsTrainer AbsRunner

快速上手 Encoder-only 与 Decoder-only快速上手

2.3 安装依赖

安装 `FlagAutoModel` 和 `FlagAutoReranker` 依赖

- 安装依赖
- 从 HuggingFace Hub 安装
- 安装依赖

2.4 安装依赖

`AbsEmbedder` 依赖

- 安装依赖 `CUDA` `NPU` `MUSA` `MPS` `CPU`
- 安装依赖
- 安装 `OOM` 依赖

3. 目录结构

```
/workspace/FlagEmbedding/
├── abc/                                # 0
│   ├── inference/                     # 0
│   │   ├── AbsEmbedder.py             # 0
│   │   ├── AbsReranker.py             # 0
│   │   └── __init__.py
│   ├── finetune/                      # 0
│   │   ├── embedder/                  # 0
│   │   ├── reranker/                  # 0
│   │   └── __init__.py
│   ├── evaluation/                    # 0
│   │   └── __init__.py
│   └── inference/                     # 0
│       ├── auto_embedder.py           # 0
│       ├── auto_reranker.py           # 0
│       ├── embedder/                  # 0
│       │   ├── encoder_only/          # Encoder-only 0
│       │   │   ├── base.py            # 0
│       │   │   └── m3.py              # BGE-M3 0
│       │   ├── decoder_only/          # Decoder-only 0
│       │   │   ├── base.py            # 0
│       │   │   ├── icl.py             # 0
│       │   │   └── pseudo_moe.py      # 0MoE 0
│       │   ├── model_mapping.py        # 0
│       │   └── __init__.py
│       ├── reranker/                  # 0
│       │   ├── encoder_only/
│       │   ├── decoder_only/
│       │   ├── model_mapping.py
│       │   └── __init__.py
│       └── __init__.py
├── finetune/                          # 0
│   ├── embedder/                      # 0
│   ├── reranker/                      # 0
│   └── __init__.py
├── evaluation/                         # 0
│   ├── mteb/                         # MTEB 0
│   ├── beir/                         # BEIR 0
│   ├── msmarco/                      # MSMARCO 0
│   ├── miracl/                       # MIRACL 0
│   ├── mkqa/                         # MKQA 0
│   ├── mldr/                         # MLDR 0
│   ├── air_bench/                    # AIR-Bench 0
│   ├── bright/                       # BRIGHT 0
│   ├── custom/                       # 0
│   └── __init__.py
├── utils/                             # 0
│   ├── transformers_compat.py         # Transformers 0
│   └── __init__.py
└── __init__.py
```

3.1 目录

目录	说明
abc/	目录
inference/	目录
finetune/	目录
evaluation/	目录
utils/	目录

4. 目录

4.1 目录

FlagEmbedding 目录

BGE 目录

- **bge-m3** 目录
- **bge-large/ base/ small-en-v1.5** 目录 v1.5 目录
- **bge-large/ base/ small-zh-v1.5** 目录 v1.5 目录
- **bge-en-icl** 目录
- **bge-multilingual-gemma2** 目录 Gemma2 目录
- **bge-code-v1** 目录
- **bge-reasoner-embed-qwen3-8b-0923** 目录

目录

- **Qwen3-Embedding** 目录 Qwen3 目录
- **E5** 目录 Microsoft E5 目录
- **GTE** 目录 GTE 目录
- **SFR** 目录 Salesforce SFR 目录
- **Linq** 目录 Linq 目录
- **BCE** 目录 BCE 目录

目录 [model_mapping.py](#) 目录

4.2 模型选择

- **bge-reranker-base** (基础模型)
- **bge-reranker-large** (大型模型)
- **bge-reranker-v2-m3** (M3 模型)
- **bge-reranker-v2-gemma** (Gemma 模型)
- **bge-reranker-v2-minicpm-layerwise** (Minicpm 逐层模型)
- **bge-reranker-v2.5-gemma2-lightweight** (轻量级 Gemma2 模型)

详细请参考 [reranker/model_mapping.py](#)

4.3 模型部署

使用 BGE 提供的 FlagEmbedding 接口

1. **BGE-M3** (基础模型)
2. **LM-Cocktail** (混合模型)
3. **LLM-Embedder** (LLM 嵌入模型)
4. **Activation-Beacon** (激活模型)

4.4 模型评估

常用数据集

- **MTEB** (多任务文本嵌入基准)
- **BEIR** (跨语言检索基准)
- **MSMARCO** (微软搜索广告相关性数据集)
- **MIRACL** (多语言检索数据集)
- **MKQA** (多语言机器问答数据集)
- **MLDR** (多语言机器阅读理解数据集)
- **AIR-Bench** (跨语言机器阅读理解基准)
- **BRIGHT** (跨语言机器阅读理解基准)

5. 应用案例

5.1 模型部署

使用 FlagAutoModel 接口

```

from FlagEmbedding import FlagAutoModel

# BGE-M3
model = FlagAutoModel.from_finetuned(
    'BAAI/bge-m3',
    normalize_embeddings=True,
    use_fp16=True
)

#
queries = [' ', ' ']
query_embeddings = model.encode_queries(queries)

#
corpus = [
    ' ',
    ' '
]
corpus_embeddings = model.encode_corpus(corpus)

```

5.2

```

from FlagEmbedding import FlagAutoReranker

#
reranker = FlagAutoReranker.from_finetuned('BAAI/bge-reranker-large')

#
scores = reranker.compute_score(
    [ (' ', ' '),
      (' ', ' ') ]
)

```

5.3

```

from FlagEmbedding import FlagAutoModel

# GPU
model = FlagAutoModel.from_finetuned(
    'BAAI/bge-large-en-v1.5',
    devices=[0, 1, 2, 3] # 4 GPU
)

#
embeddings = model.encode(large_corpus, batch_size=1024)

```

6.

- 1.
2. FlagAutoModel FlagAutoReranker

3. 选择加速方案 CUDA/NPU/MUSA/MPS/CPU 加速
4. 选择推理引擎 vllm/tensorrt-llm/gpt4o/gpt4o-mini GPU
5. 选择模型大小 7B/13B/70B/110B
6. 选择模型 BGE/E5/GTE/Qwen3 模型
7. 选择部署环境 本地/云端

7. 部署

- 本地部署 推理引擎
- 本地部署 模型
- 本地部署 推理引擎
- 本地部署 模型

02 abc layer

FlagEmbedding 模型 (ABC Layer) 模型

模型

Diagram 1

Mermaid Diagram

目录

模块	类	方法	文件
AbsEmbedder		encode/encode_queries/ encode_corpus	abc/inference/AbsEmbedder.py
AbsReranker		compute_score	abc/inference/AbsReranker.py
AbsEmbedderModel		forward/loss	abc/finetune/embedder/ AbsModeling.py
AbsRerankerModel		forward	abc/finetune/reranker/ AbsModeling.py
AbsEvalRunner		run	abc/evaluation/runner.py
AbsEvaluator		call	abc/evaluation/evaluator.py

目录

- 1. 目录
- 2. 目录
 - 2.1 AbsEmbedder
 - 2.2 AbsReranker
- 3. 目录
 - 3.1 Embedder 目录
 - 3.2 Reranker 目录
- 4. 目录
 - 4.1 AbsEvalRunner
 - 4.2 AbsEvaluator
 - 4.3 AbsEvalDataLoader
 - 4.4 目录
- 5. 目录

1. 目录

FlagEmbedding (ABC Layer) (ABC Layer)

- `embedder`: embedding 和 reranking
- `finetune`: (ABC Layer)
- `evaluation`: (ABC Layer)

encoder-only decoder-only

目录

```
abc/
├── inference/                # 
│   ├── AbsEmbedder.py
│   └── AbsReranker.py
├── finetune/                # 
│   ├── embedder/
│   │   ├── AbsModeling.py
│   │   ├── AbsDataset.py
│   │   ├── AbsTrainer.py
│   │   ├── AbsRunner.py
│   │   └── AbsArguments.py
│   └── reranker/
│       ├── AbsModeling.py
│       ├── AbsDataset.py
│       ├── AbsTrainer.py
│       ├── AbsRunner.py
│       └── AbsArguments.py
└── evaluation/              # 
    ├── runner.py
    ├── evaluator.py
    ├── data_loader.py
    ├── searcher.py
    ├── arguments.py
    └── utils.py
```

2. 目录结构

2.1 AbsEmbedder

`embedder`: [abc/inference/AbsEmbedder.py](#)

`AbsEmbedder` (ABC Layer)

□□□□

```
class AbsEmbedder(ABC):
    def __init__(
        self,
        model_name_or_path: str,
        normalize_embeddings: bool = True,
        use_fp16: bool = True,
        use_bf16: bool = False,
        query_instruction_for_retrieval: Optional[str] = None,
        query_instruction_format: str = "{}{}",
        devices: Optional[Union[str, int, List[str], List[int]]] = None,
        batch_size: int = 256,
        query_max_length: int = 512,
        passage_max_length: int = 512,
        convert_to_numpy: bool = True,
        truncate_dim: Optional[int] = None,
        **kwargs: Any,
    ):
```

□□□□:

- `normalize_embeddings`: □□□□□□□□
- `use_fp16` / `use_bf16`: □□□□□□□□
- `devices`: □□□□□ GPU/TPU/NPU/Musa □□□□□□
- `truncate_dim`: □□ Matryoshka Representation Learning (MRL) □□□□□

□□□□□□□

1. □□□□□□

```
def encode_queries(
    self,
    queries: Union[List[str], str],
    batch_size: Optional[int] = None,
    max_length: Optional[int] = None,
    convert_to_numpy: Optional[bool] = None,
    **kwargs: Any
)
```

□□□□□□□□□□□□□□ `query_instruction_for_retrieval` □

```
def encode_corpus(
    self,
    corpus: Union[List[str], str],
    batch_size: Optional[int] = None,
    max_length: Optional[int] = None,
    convert_to_numpy: Optional[bool] = None,
    **kwargs: Any
)
```

□□□□□□□□□□□□□□ `passage_instruction_for_retrieval` □

```
def encode(
    self,
    sentences: Union[List[str], str],
    batch_size: Optional[int] = None,
    max_length: Optional[int] = None,
    convert_to_numpy: Optional[bool] = None,
    instruction: Optional[str] = None,
    instruction_format: Optional[str] = None,
    **kwargs: Any
)
```

encode_queries
encode_corpus

2.

```
@abstractmethod
def encode_single_device(
    self,
    sentences: Union[List[str], str],
    batch_size: int = 256,
    max_length: int = 512,
    convert_to_numpy: bool = True,
    device: Optional[str] = None,
    **kwargs: Any,
):
    """
    pass
```

AbsEmbedder

1. [\(start_multi_process_pool\)](#)

- spawn
-
-

```

def start_multi_process_pool(
    self,
    process_target_func: Any,
) -> Dict[Literal["input", "output", "processes"], Any]:
    # CPU
    self.model.to("cpu")
    self.model.share_memory()

    # spawn
    ctx = mp.get_context("spawn")
    input_queue = ctx.Queue()
    output_queue = ctx.Queue()
    processes = []

    #
    for device_id in self.target_devices:
        p = ctx.Process(
            target=process_target_func,
            args=(device_id, self, input_queue, output_queue),
            daemon=True,
        )
        p.start()
        processes.append(p)

```

1. `encode_multi_process_worker`

- `encode_multi_process_worker`
- `encode_single_device`
- `encode_multi_process`

2. `encode_multi_process`

- `encode_multi_process`
- `encode_multi_process`
- `encode_multi_process`

```

def encode_multi_process(
    self,
    sentences: List[str],
    pool: Dict[Literal["input", "output", "processes"], Any],
    **kwargs
):
    # []
    chunk_size = math.ceil(len(sentences) / len(pool["processes"]))

    # []
    input_queue = pool["input"]
    last_chunk_id = 0
    chunk = []
    for sentence in sentences:
        chunk.append(sentence)
        if len(chunk) >= chunk_size:
            input_queue.put([last_chunk_id, chunk, kwargs])
            last_chunk_id += 1
            chunk = []

    # ... []

    # []
    results_list = sorted(
        [output_queue.get() for _ in trange(last_chunk_id)],
        key=lambda x: x[0],
    )
    # []
    return self._concatenate_results_from_multi_process([result[1] for result in results_list])

```

1. [] [] [] []

- `stop_self_pool`: [] [] [] [] GPU [] []
- `_del`: [] [] [] [] [] [] [] []

[] [] [] [] [] []

[get_target_devices](#) [] [] [] [] [] [] [] [] [] []

```

@staticmethod
def get_target_devices(devices: Union[str, int, List[str], List[int]]) -> List[str]:
    if devices is None:
        # []
        if torch.cuda.is_available():
            return [f"cuda:{i}" for i in range(torch.cuda.device_count())]
        elif is_torch_npu_available():
            return [f"npu:{i}" for i in range(torch.npu.device_count())]
        elif hasattr(torch, "musa") and torch.musa.is_available():
            return [f"musa:{i}" for i in range(torch.musa.device_count())]
        elif torch.backends.mps.is_available():
            # ... MPS []
        else:
            return ["cpu"]
    # ... []

```

支持的硬件

- NVIDIA CUDA GPU
- Huawei NPU
- Moore Threads Musa
- Apple MPS
- CPU

主要接口

1. `get_detailed_instruct`

```
python @staticmethod def get_detailed_instruct(instruction_format: str, instruction: str, sentence: str): """ 如果 instruction_format 中包含换行符，则将其替换为转义字符。 """ if "\\n" in instruction_format: instruction_format = instruction_format.replace("\\n", "\n") return instruction_format.format(instruction, sentence)
```

2. `_concatenate_results_from_multi_process`

- 支持 torch.Tensor 和 numpy.ndarray

3. `NumPy` `_convert_to_numpy`

- 支持 bf16 和 NumPy 的 bf16
- bf16 → float32

4. `_truncate_embeddings`

- 支持 MRL 和 1D 向量

2.2 AbsReranker

文件路径: [abc/inference/AbsReranker.py](#)

`AbsReranker` 是一个抽象基类，用于定义重排接口。

□□□□

```
class AbsReranker(ABC):
    def __init__(
        self,
        model_name_or_path: str,
        use_fp16: bool = False,
        query_instruction_for_rerank: Optional[str] = None,
        query_instruction_format: str = "{}{}",
        passage_instruction_for_rerank: Optional[str] = None,
        passage_instruction_format: str = "{}{}",
        devices: Optional[Union[str, int, List[str], List[int]]] = None,
        batch_size: int = 128,
        query_max_length: Optional[int] = None,
        max_length: int = 512,
        normalize: bool = False,
        **kwargs: Any,
    ):
```

□□□□:

- □□□□□□□□□□□□
- □□□□□□□□□□ AbsEmbedder □□□
- □□□□□□□□

□□□□

1. □□□□

```
def compute_score(
    self,
    sentence_pairs: Union[List[Tuple[str, str]], Tuple[str, str]],
    **kwargs
):
    """□"""
    if isinstance(sentence_pairs[0], str):
        sentence_pairs = [sentence_pairs]

    # □
    sentence_pairs = self.get_detailed_inputs(sentence_pairs)

    # □
    if isinstance(sentence_pairs, str) or len(self.target_devices) == 1:
        return self.compute_score_single_gpu(...)
    else:
        # □
        if self.pool is None:
            self.pool = self.start_multi_process_pool()
        return self.encode_multi_process(...)
```

2. `compute_score_single_gpu`

```
@abstractmethod
def compute_score_single_gpu(
    self,
    sentence_pairs: Union[List[Tuple[str, str]], Tuple[str, str]],
    batch_size: int = 256,
    query_max_length: Optional[int] = None,
    max_length: int = 512,
    normalize: bool = False,
    device: Optional[str] = None,
    **kwargs: Any,
):
    """compute_score_single_gpu
    pass
```

`get_detailed_inputs`

`get_detailed_inputs` `get_detailed_inputs`

```
def get_detailed_inputs(self, sentence_pairs: Union[str, List[str]]):
    """get_detailed_inputs
    if isinstance(sentence_pairs, str):
        sentence_pairs = [sentence_pairs]

    if self.query_instruction_for_rerank is not None:
        if self.passage_instruction_for_rerank is None:
            return [
                [
                    self.get_detailed_instruct(self.query_instruction_format,
                                                self.query_instruction_for_rerank,
                                                sentence_pair[0]),
                    sentence_pair[1]
                ] for sentence_pair in sentence_pairs
            ]
        else:
            return [
                [
                    self.get_detailed_instruct(self.query_instruction_format,
                                                self.query_instruction_for_rerank,
                                                sentence_pair[0]),
                    self.get_detailed_instruct(self.passage_instruction_format,
                                                self.passage_instruction_for_rerank,
                                                sentence_pair[1])
                ] for sentence_pair in sentence_pairs
            ]
    # ... get_detailed_inputs
```

`get_detailed_inputs`

`AbsEmbedder` `get_detailed_inputs`

- `start_multi_process_pool`
- `_encode_multi_process_worker`
- `encode_multi_process`

3. Embedder

3.1 Embedder

3.1.1 AbsEmbedderModel (API)

API: [abc/finetune/embedder/AbsModeling.py](https://github.com/abcfinetune/finetune/blob/master/finetune/embedder/AbsModeling.py)

Embedder API

API

```
@dataclass
class EmbedderOutput(ModelOutput):
    q_reps: Optional[Tensor] = None
    p_reps: Optional[Tensor] = None
    loss: Optional[Tensor] = None
    scores: Optional[Tensor] = None
```

API

```
class AbsEmbedderModel(ABC, nn.Module):
    def __init__(
        self,
        base_model,
        tokenizer: PreTrainedTokenizer = None,
        negatives_cross_device: bool = False,
        temperature: float = 1.0,
        sub_batch_size: int = -1,
        kd_loss_type: str = 'kl_div',
        use_mrl: bool = False,
        mrl_dims: List[int] = [],
    ):
```

API:

- `negatives_cross_device`: `bool`
- `temperature`: `float`
- `kd_loss_type`: `str` (`kl_div` or `m3_kd_loss`)
- `use_mrl`: `bool` Matryoshka Representation Learning
- `mrl_dims`: MRL `list`

□□□□

```
@abstractmethod
def encode(self, features):
    """□"""
    pass

@abstractmethod
def compute_loss(self, scores, target):
    """□"""
    pass

@abstractmethod
def compute_score(self, q_reps, p_reps):
    """□□"""
    pass

@abstractmethod
def save(self, output_dir: str):
    """□"""
    pass
```

□□□□□□

□□□□□□□□□□□□□□□□

1. □□□□□□ ([_compute_no_in_batch_neg_loss](#))

```
def _compute_no_in_batch_neg_loss(self, q_reps, p_reps, teacher_targets=None, ...):
    """□"""
    group_size = p_reps.size(0) // q_reps.size(0)

    # □
    local_scores = self.compute_local_score(q_reps, p_reps, ...)

    if teacher_targets is not None:
        # □
        loss = self.distill_loss(self.kd_loss_type, teacher_targets, local_scores, group_size=group_size)
        if self.kd_loss_type == "kl_div":
            # □
            local_targets = torch.zeros(local_scores.size(0), ...)
            loss += self.compute_loss(local_scores, local_targets)
    else:
        local_targets = torch.zeros(local_scores.size(0), ...)
        loss = self.compute_loss(local_scores, local_targets)

    return local_scores, loss
```

2. `compute_in_batch_neg_loss`

```
def _compute_in_batch_neg_loss(self, q_reps, p_reps, teacher_targets=None, ...):
    """
    group_size = p_reps.size(0) // q_reps.size(0)

    #
    if compute_score_func is None:
        scores = self.compute_score(q_reps, p_reps) # (batch_size, batch_size * group_size)
    else:
        scores = compute_score_func(q_reps, p_reps, ...)

    if teacher_targets is not None:
        if self.kd_loss_type == "kl_div":
            #
            student_scores = self.get_local_score(q_reps, p_reps, scores)
            loss = self.distill_loss(...)
            #
            idxs = torch.arange(q_reps.size(0), ...)
            targets = idxs * group_size
            loss += self.compute_loss(scores, targets)
        elif self.kd_loss_type == "m3_kd_loss":
            # M3
            loss = self.distill_loss(...)
        else:
            idxs = torch.arange(q_reps.size(0), ...)
            targets = idxs * group_size #
            loss = self.compute_loss(scores, targets)

    return scores, loss
```

3. `compute_cross_device_neg_loss`

```
def _compute_cross_device_neg_loss(self, q_reps, p_reps, teacher_targets=None, ...):
    """
    group_size = p_reps.size(0) // q_reps.size(0)

    #
    cross_q_reps = self._dist_gather_tensor(q_reps) # (world_size * batch_size, dim)
    cross_p_reps = self._dist_gather_tensor(p_reps) # (world_size * batch_size * group_size, dim)

    #
    if compute_score_func is None:
        cross_scores = self.compute_score(cross_q_reps, cross_p_reps)
    else:
        cross_scores = compute_score_func(cross_q_reps, cross_p_reps, ...)

    # ...
    return cross_scores, loss
```

~~~~~

[\\_dist\\_gather\\_tensor](#) []

```

def _dist_gather_tensor(self, t: Optional[torch.Tensor]):
    """ """
    if t is None:
        return None
    t = t.contiguous()

    #
    all_tensors = [torch.empty_like(t) for _ in range(self.world_size)]

    #
    dist.all_gather(all_tensors, t)

    #
    all_tensors[self.process_rank] = t

    #
    all_tensors = torch.cat(all_tensors, dim=0)

    return all_tensors

```

□□□□□□

[distill\\_loss](#) □□□□□□

```

@staticmethod
def distill_loss(kd_loss_type, teacher_targets, student_scores, group_size=None):
    if kd_loss_type == 'kl_div':
        # KL
        return -torch.mean(
            torch.sum(torch.log_softmax(student_scores, dim=-1) * teacher_targets, dim=-1)
        )
    elif kd_loss_type == 'm3_kd_loss':
        # BGE-M3
        labels = torch.arange(student_scores.size(0), ...)
        labels = labels * group_size
        loss = 0
        mask = torch.zeros_like(student_scores)
        for i in range(group_size):
            temp_target = labels + i
            temp_scores = student_scores + mask
            temp_loss = F.cross_entropy(temp_scores, temp_target, reduction="none")
            loss += torch.mean(teacher_targets[:, i] * temp_loss)
        #
        mask = torch.scatter(mask, dim=-1, index=temp_target.unsqueeze(-1),
                             value=torch.finfo(student_scores.dtype).min)
    return loss

```

□□□□

[forward](#) □□□□□□□□

```

def forward(
    self,
    queries: Union[Dict[str, Tensor], List[Dict[str, Tensor]]] = None,
    passages: Union[Dict[str, Tensor], List[Dict[str, Tensor]]] = None,
    teacher_scores: Union[None, List[float]] = None,
    no_in_batch_neg_flag: bool = False,
):
    # []
    q_reps = self.encode(queries)
    p_reps = self.encode(passages)

    if self.training:
        # []
        if teacher_scores is not None:
            teacher_scores = torch.tensor(teacher_scores, ...)
            teacher_scores = teacher_scores.view(...)
            teacher_targets = F.softmax(teacher_scores, dim=-1)
        else:
            teacher_targets = None

        # []
        if no_in_batch_neg_flag:
            compute_loss_func = self._compute_no_in_batch_neg_loss
        else:
            if self.negatives_cross_device:
                compute_loss_func = self._compute_cross_device_neg_loss
            else:
                compute_loss_func = self._compute_in_batch_neg_loss

        # MRL []
        if self.use_mrl:
            all_loss = torch.tensor(0.0, ...)
            # []
            for dim_q_reps, dim_p_reps in zip(q_reps, p_reps):
                _, mrl_loss = compute_loss_func(dim_q_reps, dim_p_reps, teacher_targets=teacher_targets)
                all_loss += mrl_loss
            loss = all_loss / len(self.mrl_dims)
        else:
            scores, loss = compute_loss_func(q_reps, p_reps, teacher_targets=teacher_targets)
    else:
        loss = None

    return EmbedderOutput(loss=loss)

```

□□□□

#### 1. □□□□□ (get\_local\_score)

```

python def get_local_score(self, q_reps, p_reps, all_scores): """[]" group_size =
p_reps.size(0) // q_reps.size(0) indices = torch.arange(0, q_reps.size(0), ...) *
group_size specific_scores = [] for i in range(group_size):
specific_scores.append( all_scores[torch.arange(q_reps.size(0), ...), indices + i] )
return torch.stack(specific_scores, dim=1).view(q_reps.size(0), -1)

```

### 3.1.2 AbsEmbedderTrainDataset (数据集)

位置: [abc/finetune/embedder/AbsDataset.py](https://github.com/abcfinetune/finetune/blob/master/finetune/embedder/AbsDataset.py)

数据集

```
class AbsEmbedderTrainDataset(Dataset):
    def __init__(
        self,
        args: AbsEmbedderDataArguments,
        tokenizer: PreTrainedTokenizer
    ):
        self.args = args
        self.tokenizer = tokenizer
        self.shuffle_ratio = args.shuffle_ratio

        # 训练数据集
        train_datasets = []
        for data_dir in args.train_data:
            # ... json/jsonl 文件
            temp_dataset = self._load_dataset(data_dir)
            train_datasets.append(temp_dataset)
        self.dataset = datasets.concatenate_datasets(train_datasets)
```

数据集 (数据集):

```
{
    "query": "问题",
    "pos": ["正面", "负面"],
    "neg": ["正面", "负面"],
    "pos_scores": [0.9, 0.8], // 正面
    "neg_scores": [0.3, 0.2] // 负面
}
```



getitem (getitem)

```
def __getitem__(self, item):
    data = self.dataset[item]
    train_group_size = self.args.train_group_size

    query = data['query']
    # []
    if self.args.query_instruction_for_retrieval is not None:
        query = self.args.query_instruction_format.format(...)

    passages = []
    teacher_scores = []

    # []
    pos_idx = random.choice(list(range(len(data['pos']))))
    passages.append(self._shuffle_text(data['pos'][pos_idx]))

    # []
    neg_all_idx = list(range(len(data['neg'])))
    if len(data['neg']) < train_group_size - 1:
        num = math.ceil((train_group_size - 1) / len(data['neg']))
        neg_idx = random.sample(neg_all_idx * num, train_group_size - 1)
    else:
        neg_idx = random.sample(neg_all_idx, self.args.train_group_size - 1)

    for neg_idx in neg_idx:
        passages.append(data['neg'][neg_idx])

    # []
    if self.args.knowledge_distillation:
        teacher_scores.append(data['pos_scores'][pos_idx])
        for neg_idx in neg_idx:
            teacher_scores.append(data['neg_scores'][neg_idx])

    # []
    if self.args.passage_instruction_for_retrieval is not None:
        passages = [self.args.passage_instruction_format.format(...) for p in passages]

    return query, passages, teacher_scores
```

shuffle\_text

[\\_shuffle\\_text](#) shuffle\_text

```
def _shuffle_text(self, text):
    """
    if self.shuffle_ratio > 0 and len(text) > 100 and random.random() < self.shuffle_ratio:
        split_text = []
        chunk_size = len(text)//3 + 1
        for i in range(0, len(text), chunk_size):
            split_text.append(text[i:i+chunk_size])
        random.shuffle(split_text)
        return " ".join(split_text)
    else:
        return text
```

数据集 (AbsEmbedderSameDatasetTrainDataset)

数据集 batch 数据集

```
class AbsEmbedderSameDatasetTrainDataset(AbsEmbedderTrainDataset):
    def __init__(
        self,
        args: AbsEmbedderDataArguments,
        default_batch_size: int,
        seed: int,
        tokenizer: PreTrainedTokenizer,
        process_index: int = 0,
        num_processes: int = 1
    ):
        # ...
        # batch size
        batch_size_idxes.append(self._get_file_batch_size(temp_dataset, default_batch_size))
        # no_in_batch_neg
        no_in_batch_neg_flags.append(no_in_batch_neg_flag)

        # batch
        self.refresh_epoch()

    def refresh_epoch(self):
        """epoch batch"""
        self.deterministic_generator.shuffle(self.datasets_inxs)
        batch_datas = []
        for dataset_inx in self.datasets_inxs:
            self.deterministic_generator.shuffle(self.each_data_idxes[dataset_inx])
            cur_batch_size = self.batch_size_idxes[dataset_inx] * self.num_processes
            # ... batch
            self.deterministic_generator.shuffle(batch_datas)
            self.batch_datas = batch_datas
```

数据集:

- 数据集 batch size
- 数据集
- EmbedderTrainerCallbackForDataRefresh 数据集 epoch 数据集

## DataCollator

```
@dataclass
class AbsEmbedderCollator(DataCollatorWithPadding):
    query_max_len: int = 32
    passage_max_len: int = 128
    sub_batch_size: int = -1

    def __call__(self, features):
        queries = [f[0] for f in features]
        passages = [f[1] for f in features]
        teacher_scores = [f[2] for f in features]

        # []
        if isinstance(queries[0], list):
            queries = sum(queries, [])
        if isinstance(passages[0], list):
            passages = sum(passages, [])

        # tokenize
        queries_inputs = self.tokenizer(queries, truncation=True, max_length=self.query_max_len)
        passages_inputs = self.tokenizer(passages, truncation=True, max_length=self.passage_max_len)

        # []
        if self.sub_batch_size is None or self.sub_batch_size <= 0:
            q_collated = self.tokenizer.pad(queries_inputs, ...)
            d_collated = self.tokenizer.pad(passages_inputs, ...)
        else:
            # []
            q_collated = []
            for i in range(0, len(queries_inputs['attention_mask']), self.sub_batch_size):
                # ... pad []
                d_collated = []
                # ... []

        return {
            "queries": q_collated,
            "passages": d_collated,
            "teacher_scores": teacher_scores,
            "no_in_batch_neg_flag": False
        }
```

---

### 3.1.3 AbsEmbedderTrainer (📄📄📄📄)

📄📄📄: [abc/finetune/embedder/AbsTrainer.py](https://github.com/abcfinetune/finetune/blob/master/finetune/embedder/AbsTrainer.py)

```

class AbsEmbedderTrainer(ABC, Trainer):
    @abstractmethod
    def _save(self, output_dir: Optional[str] = None, state_dict=None):
        """
        pass

    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        """Trainer"""
        outputs = model(**inputs)
        loss = outputs.loss
        return (loss, outputs) if return_outputs else loss

```

□□□□□□□□□□□□□□□□□□□□

### 3.1.4 AbsEmbedderRunner (□□□□□)

□□□□: <abc/finetune/embedder/AbsRunner.py>

□□□□□□□□□□

```

class AbsEmbedderRunner(ABC):
    def __init__(
        self,
        model_args: AbsEmbedderModelArguments,
        data_args: AbsEmbedderDataArguments,
        training_args: AbsEmbedderTrainingArguments
    ):
        # ...
        self.tokenizer, self.model = self.load_tokenizer_and_model()
        self.train_dataset = self.load_train_dataset()
        self.data_collator = self.load_data_collator()
        self.trainer = self.load_trainer()

    @abstractmethod
    def load_tokenizer_and_model(self) -> Tuple[PreTrainedTokenizer, AbsEmbedderModel]:
        pass

    @abstractmethod
    def load_trainer(self) -> AbsEmbedderTrainer:
        pass

    def load_train_dataset(self) -> AbsEmbedderTrainDataset:
        """
        if self.data_args.same_dataset_within_batch:
            return AbsEmbedderSameDatasetTrainDataset(...)
        else:
            return AbsEmbedderTrainDataset(...)

    def load_data_collator(self) -> AbsEmbedderCollator:
        """Collator"""
        if self.data_args.same_dataset_within_batch:
            return AbsEmbedderSameDatasetCollator(...)
        else:
            return AbsEmbedderCollator(...)

    def run(self):
        """
        Path(self.training_args.output_dir).mkdir(parents=True, exist_ok=True)
        self.trainer.train(resume_from_checkpoint=self.training_args.resume_from_checkpoint)
        self.trainer.save_model()

```

### 3.1.5 AbsEmbedderArguments (□□□□)

□□□□: <abc/finetune/embedder/AbsArguments.py>

□□□□□□□□

#### 1. **AbsEmbedderModelArguments:** □□□□

- model\_name\_or\_path
- config\_name

- `tokenizer_name`

- `[]`

## 2. **AbsEmbedderDataArguments:** `[] [] [] []`

- `train_data` : `[] [] [] [] [] [] [] []`

- `train_group_size` : `[] [] [] [] [] [] [] [] [] 1 [] + (n-1) [] []`

- `query_max_len` / `passage_max_len`

- `knowledge_distillation` : `[] [] [] []` KD

- `same_dataset_within_batch` : `[] [] [] [] [] []` batch

- `shuffle_ratio` : `[] [] [] [] [] []`

- `[]`

## 3. **AbsEmbedderTrainingArguments:** `[] [] [] [] [] []` transformers.TrainingArguments`[]`

- `negatives_cross_device` : `[] [] [] [] [] []`

- `temperature` : `[] [] [] []`

- `sentence_pooling_method` : `[] [] [] []` ( `cls` / `mean` / `last_token` )

- `normalize_embeddings` : `[] [] [] [] [] []`

- `kd_loss_type` : `[] [] [] [] [] []`

- `use_mrl` / `mrl_dims` : MRL `[] []`

- `[]`

---

## 3.2 Reranker `[] [] [] [] [] []`

Reranker `[] [] [] [] [] [] [] []` Embedder `[] [] [] [] [] [] [] []`

### 3.2.1 AbsRerankerModel

`[] [] [] []`: [abc/finetune/reranker/AbsModeling.py](https://github.com/abcfinetune/finetune/blob/master/finetune/reranker/AbsModeling.py)

```

@dataclass
class RerankerOutput(ModelOutput):
    loss: Optional[Tensor] = None
    scores: Optional[Tensor] = None

class AbsRerankerModel(ABC, nn.Module):
    def __init__(
        self,
        base_model,
        tokenizer: PreTrainedTokenizer = None,
        train_batch_size: int = 4,
    ):
        nn.Module.__init__(self)
        self.model = base_model
        self.tokenizer = tokenizer
        self.cross_entropy = nn.CrossEntropyLoss(reduction='mean')

        # [pad token
        if self.model.config.pad_token_id is None:
            self.model.config.pad_token_id = self.tokenizer.pad_token_id
        self.config = self.model.config

        self.train_batch_size = train_batch_size

        # [decoder-only [Yes [
        self.yes_loc = self.tokenizer('Yes', add_special_tokens=False)['input_ids'][-1]

    @abstractmethod
    def encode(self, features):
        pass

    def forward(self, pair: Union[Dict[str, Tensor], List[Dict[str, Tensor]]] = None,
                teacher_scores: Optional[Tensor] = None):
        ranker_logits = self.encode(pair)

        if self.training:
            # [(batch_size, group_size)
            grouped_logits = ranker_logits.view(self.train_batch_size, -1)
            target = torch.zeros(self.train_batch_size, ..., dtype=torch.long)
            loss = self.compute_loss(grouped_logits, target)

            # [
            if teacher_scores is not None:
                teacher_scores = torch.Tensor(teacher_scores)
                teacher_targets = teacher_scores.view(...)
                teacher_targets = torch.softmax(teacher_targets.detach(), dim=-1)
                teacher_targets = teacher_targets.to(...)
                loss += -torch.mean(
                    torch.sum(torch.log_softmax(grouped_logits, dim=-1) * teacher_targets, dim=-1)
                )
            else:
                loss = None

        return RerankerOutput(loss=loss, scores=ranker_logits)

    def compute_loss(self, scores, target):
        return self.cross_entropy(scores, target)

```

□□□□:

- □ Embedder □□□□□□□□
  - □□□□□□□□□□
  - □□□□□
- 

### 3.2.2 □□ Reranker □□

- **AbsRerankerTrainDataset:** [abc/finetune/reranker/AbsDataset.py](#)
  - □□ Embedder□□□□ query-passage □□
  - □□ AbsLLMRerankerTrainDataset □□ decoder-only □□
  - **AbsRerankerTrainer:** [abc/finetune/reranker/AbsTrainer.py](#)
  - **AbsRerankerRunner:** [abc/finetune/reranker/AbsRunner.py](#)
  - **AbsRerankerArguments:** [abc/finetune/reranker/AbsArguments.py](#)
  - □□□ model\_type □□□□ encoder/decoder
  - □□□ sep\_token □□ LLM □□
- 

## 4. □□□□□□

---

### 4.1 AbsEvalRunner

□□□□: [abc/evaluation/runner.py](#)

□□□□□□□□□□□□□□□□



```

class AbsEvalRunner:
    def __init__(
        self,
        eval_args: AbsEvalArgs,
        model_args: AbsEvalModelArgs,
    ):
        self.eval_args = eval_args
        self.model_args = model_args

        # []
        self.retriever, self.reranker = self.load_retriever_and_reranker()
        self.data_loader = self.load_data_loader()
        self.evaluator = self.load_evaluator()

    @staticmethod
    def get_models(model_args: AbsEvalModelArgs) -> Tuple[AbsEmbedder, Union[AbsReranker, None]]:
        """[]Embedder []Reranker"""
        embedder = FlagAutoModel.from_finetuned(...)
        embedder.model.config._name_or_path = model_args.embedder_name_or_path

        reranker = None
        if model_args.reranker_name_or_path is not None:
            reranker = FlagAutoReranker.from_finetuned(...)
            reranker.model.config._name_or_path = model_args.reranker_name_or_path

        return embedder, reranker

    def run(self):
        """[]"""
        if self.eval_args.dataset_names is None:
            dataset_names = self.data_loader.available_dataset_names()
        else:
            dataset_names = self.data_loader.check_dataset_names(self.eval_args.dataset_names)

        if len(dataset_names) == 0:
            # []
            self.evaluator(
                splits=self.eval_args.splits,
                search_results_save_dir=self.eval_args.output_dir,
                retriever=self.retriever,
                reranker=self.reranker,
                corpus_embd_save_dir=self.eval_args.corpus_embd_save_dir,
                ignore_identical_ids=self.eval_args.ignore_identical_ids,
                k_values=self.eval_args.k_values
            )
        else:
            # []
            for dataset_name in dataset_names:
                self.evaluator(
                    splits=self.eval_args.splits,
                    search_results_save_dir=self.eval_args.output_dir,
                    retriever=self.retriever,
                    reranker=self.reranker,
                    corpus_embd_save_dir=self.eval_args.corpus_embd_save_dir,
                    ignore_identical_ids=self.eval_args.ignore_identical_ids,
                    k_values=self.eval_args.k_values,
                    dataset_name=dataset_name,
                )

```

```
# []
self.evaluate_metrics(
    search_results_save_dir=self.eval_args.output_dir,
    output_method=self.eval_args.eval_output_method,
    output_path=self.eval_args.eval_output_path,
    metrics=self.eval_args.eval_metrics
)
```

---

## 4.2 AbsEvaluator

□□□□: [abc/evaluation/evaluator.py](#)

□□□□□□□□□□□□□□□□□□□□

```

class AbsEvaluator:
    def __init__(
        self,
        eval_name: str,
        data_loader: AbsEvalDataLoader,
        overwrite: bool = False,
    ):
        self.eval_name = eval_name
        self.data_loader = data_loader
        self.overwrite = overwrite

    def __call__(
        self,
        splits: Union[str, List[str]],
        search_results_save_dir: str,
        retriever: EvalRetriever,
        reranker: Optional[EvalReranker] = None,
        corpus_embd_save_dir: Optional[str] = None,
        ignore_identical_ids: bool = False,
        k_values: List[int] = [1, 3, 5, 10, 100, 1000],
        dataset_name: Optional[str] = None,
        **kwargs,
    ):
        # 1. splits
        checked_splits = self.data_loader.check_splits(splits, dataset_name=dataset_name)

        # 2.
        no_reranker_search_results_save_dir = os.path.join(
            search_results_save_dir, str(retriever), "NoReranker"
        )
        os.makedirs(no_reranker_search_results_save_dir, exist_ok=True)

        flag = False #
        for split in checked_splits:
            save_path = os.path.join(no_reranker_search_results_save_dir, save_name)
            if not os.path.exists(save_path) or self.overwrite:
                flag = True
                break

        no_reranker_search_results_dict = {}
        if flag:
            #
            corpus = self.data_loader.load_corpus(dataset_name=dataset_name)
            queries_dict = {split: self.data_loader.load_queries(...) for split in checked_splits}
            all_queries = {k: v for d in queries_dict.values() for k, v in d.items()}

            all_no_reranker_search_results = retriever(
                corpus=corpus,
                queries=all_queries,
                corpus_embd_save_dir=corpus_embd_save_dir,
                ignore_identical_ids=ignore_identical_ids,
                **kwargs,
            )

            #
            for split in checked_splits:
                no_reranker_search_results_dict[split] = {
                    qid: all_no_reranker_search_results[qid]

```



```

class AbsEvalDataLoader(ABC):
    def __init__(
        self,
        eval_name: str,
        dataset_dir: Optional[str] = None,
        cache_dir: Optional[str] = None,
        token: Optional[str] = None,
        force_redownload: bool = False
    ):
        self.eval_name = eval_name
        self.dataset_dir = dataset_dir
        self.cache_dir = os.path.join(cache_dir, eval_name) if cache_dir else ...
        self.force_redownload = force_redownload

    def available_dataset_names(self) -> List[str]:
        return []

    @abstractmethod
    def available_splits(self, dataset_name: Optional[str] = None) -> List[str]:
        pass

    def load_corpus(self, dataset_name: Optional[str] = None):
        """
        if self.dataset_dir is not None:
            return self._load_local_corpus(...)
        else:
            return self._load_remote_corpus(...)

    def load_queries(self, dataset_name: Optional[str] = None, split: str = 'test'):
        """
        # ...

    def load_qrels(self, dataset_name: Optional[str] = None, split: str = 'test'):
        """qrels ("""
        # ...

    #
    def _load_remote_corpus(self, ...):
        raise NotImplementedError
    def _load_remote_queries(self, ...):
        raise NotImplementedError
    def _load_remote_qrels(self, ...):
        raise NotImplementedError

```

数据集 (数据集):

- corpus.jsonl: {"id": "...", "title": "...", "text": "..."}
- {split}\_queries.jsonl: {"id": "...", "text": "..."}
- {split}\_qrels.jsonl: {"qid": "...", "docid": "...", "relevance": 1}

## 4.4 数据集

数据集: <abc/evaluation/searcher.py>

## EvalRetriever (eval.py)

```
class EvalRetriever(ABC):
    def __init__(self, embedder: AbsEmbedder, search_top_k: int = 1000, overwrite: bool = False):
        self.embedder = embedder
        self.search_top_k = search_top_k
        self.overwrite = overwrite

    def __str__(self) -> str:
        return os.path.basename(self.embedder.model.config._name_or_path)

    def stop_multi_process_pool(self):
        self.embedder.stop_self_pool()

    @abstractmethod
    def __call__(
        self,
        corpus: Dict[str, Dict[str, Any]],
        queries: Dict[str, str],
        corpus_embd_save_dir: Optional[str] = None,
        ignore_identical_ids: bool = False,
        **kwargs,
    ) -> Dict[str, Dict[str, float]]:
        pass
```



```

class EvalDenseRetriever(EvalRetriever):
    def __call__(
        self,
        corpus: Dict[str, Dict[str, Any]],
        queries: Dict[str, str],
        corpus_embd_save_dir: Optional[str] = None,
        ignore_identical_ids: bool = False,
        **kwargs,
    ):
        # 1. []
        corpus_ids = []
        corpus_texts = []
        for docid, doc in corpus.items():
            corpus_ids.append(docid)
            corpus_texts.append(doc["text"] if "title" not in doc
                                else f"{doc['title']} {doc['text']}".strip())

        queries_ids = []
        queries_texts = []
        for qid, query in queries.items():
            queries_ids.append(qid)
            queries_texts.append(query)

        # 2. []
        if corpus_embd_save_dir is not None:
            if os.path.exists(os.path.join(corpus_embd_save_dir, "doc.npy")) and not self.overwrite:
                corpus_emb = np.load(os.path.join(corpus_embd_save_dir, "doc.npy"))
            else:
                corpus_emb = self.embedder.encode_corpus(corpus_texts, **kwargs)
        else:
            corpus_emb = self.embedder.encode_corpus(corpus_texts, **kwargs)

        queries_emb = self.embedder.encode_queries(queries_texts, **kwargs)

        # []M3 []dict []
        if isinstance(corpus_emb, dict):
            corpus_emb = corpus_emb["dense_vecs"]
        if isinstance(queries_emb, dict):
            queries_emb = queries_emb["dense_vecs"]

        # []
        if corpus_embd_save_dir is not None and ...:
            os.makedirs(corpus_embd_save_dir, exist_ok=True)
            np.save(os.path.join(corpus_embd_save_dir, "doc.npy"), corpus_emb)

        # 3. []FAISS []
        faiss_index = index(corpus_embeddings=corpus_emb)
        all_scores, all_indices = search(query_embeddings=queries_emb,
                                         faiss_index=faiss_index,
                                         k=self.search_top_k)

        # 4. []
        results = {}
        for idx, (scores, indices) in enumerate(zip(all_scores, all_indices)):
            results[queries_ids[idx]] = {}
            for score, indice in zip(scores, indices):
                if indice != -1:
                    if ignore_identical_ids and corpus_ids[indice] == queries_ids[idx]:

```



```

        continue
        results[queries_ids[idx]][corpus_ids[indice]] = float(score)

    return results

```

## EvalReranker

```

class EvalReranker:
    def __init__(self, reranker: AbsReranker, rerank_top_k: int = 100):
        self.reranker = reranker
        self.rerank_top_k = rerank_top_k

    def __call__(
        self,
        corpus: Dict[str, Dict[str, Any]],
        queries: Dict[str, str],
        search_results: Dict[str, Dict[str, float]],
        ignore_identical_ids: bool = False,
        **kwargs,
    ):
        # 1. []rerank_top_k
        for qid in search_results:
            search_results[qid] = dict(
                sorted(search_results[qid].items(), key=lambda x: x[1], reverse=True)[:self.rerank_top_k]
            )

        # 2. []
        sentence_pairs = []
        pairs = []
        for qid in search_results:
            for docid in search_results[qid]:
                if ignore_identical_ids and qid == docid:
                    continue
                doc = corpus[docid]
                doc_text = doc["text"] if "title" not in doc else f"{doc['title']} {doc['text']}".strip()
                sentence_pairs.append({"qid": qid, "docid": docid, "query": queries[qid], "doc": doc_text})
                pairs.append((queries[qid], doc_text))

        # 3. []
        scores = self.reranker.compute_score(pairs)
        for i, score in enumerate(scores):
            sentence_pairs[i]["score"] = float(score)

        # 4. []
        reranked_results = {qid: {} for qid in search_results}
        for pair in sentence_pairs:
            reranked_results[pair["qid"]][pair["docid"]] = pair["score"]

        return reranked_results

```

---

## 4.5 utils (utils)

utils: [abc/evaluation/utils.py](#)

utils FAISS utils

utils

1. **evaluate\_metrics**: utilsNDCGMAPRecallPrecision
  - `pytrec_eval`
  - cut-off
2. **evaluate\_mrr**: Mean Reciprocal Rank
3. **evaluate\_recall\_cap**: Capped Recall

## FAISS

```
def index(
    index_factory: str = "Flat",
    corpus_embeddings: Optional[np.ndarray] = None,
    load_path: Optional[str] = None,
    device: Optional[str] = None
):
    """FAISS"""
    #
    faiss_index = faiss.index_factory(corpus_embeddings.shape[-1], index_factory, faiss.METRIC_INNER_PRODUCT)

    # GPU
    if device is None and torch.cuda.is_available():
        try:
            co = faiss.GpuMultipleClonerOptions()
            co.shard = True
            co.useFloat16 = True
            faiss_index = faiss.index_cpu_to_all_gpus(faiss_index, co)
        except:
            print('faiss do not support GPU, please uninstall faiss-cpu, faiss-gpu and install faiss-gpu')

    #
    faiss_index.train(corpus_embeddings)
    faiss_index.add(corpus_embeddings)

    return faiss_index


def search(
    faiss_index: faiss.Index,
    k: int = 100,
    query_embeddings: Optional[np.ndarray] = None,
    load_path: Optional[str] = None
):
    """"""
    all_scores = []
    all_indices = []
    # OOM
    for i in tqdm(range(0, len(query_embeddings), 32), desc="Searching"):
        j = min(i + 32, len(query_embeddings))
        score, indice = faiss_index.search(query_embeddings[i:j].astype(np.float32), k=k)
        all_scores.append(score)
        all_indices.append(indice)
    return np.concatenate(all_scores, axis=0), np.concatenate(all_indices, axis=0)
```

---

## 4.6

: [abc/evaluation/arguments.py](#)

### 1. AbsEvalArgs:

- eval\_name:

- `dataset_dir`: 数据集目录
- `dataset_names`: 数据集名称
- `splits`: 数据集划分
- `output_dir`: 输出目录
- `search_top_k` / `rerank_top_k`
- `k_values`: 切分 cut-off
- `eval_metrics`: 评估指标
- 

## 2. `AbsEvalModelArgs`: 模型参数

- `embedder_name_or_path` / `embedder_model_class`
- `reranker_name_or_path` / `reranker_model_class`
- 批量大小 batch size / 最大长度 max length
- 

# 5. 模型实现

## 5.1 接口

### 1. 模板方法 (Template Method)

接口: 定义

- `AbsEmbedder` / `AbsReranker` / `AbsEmbedderModel` 接口
- `encode` 方法
- `encode_single_device` 方法

实现:

```
# AbsEmbedder 实现
def encode(self, sentences, ...):
    # 处理输入
    if instruction is not None:
        sentences = [self.get_detailed_instruct(...) for ...]

    if isinstance(sentences, str) or len(self.target_devices) == 1:
        return self.encode_single_device(...) # 单设备
    else:
        return self.encode_multi_process(...)
```

### 2. 策略 (Strategy)

接口: 定义

- `AbsEmbedderModel` 接口

```
if no_in_batch_neg_flag:
    compute_loss_func = self._compute_no_in_batch_neg_loss
else:
    if self.negatives_cross_device:
        compute_loss_func = self._compute_cross_device_neg_loss
    else:
        compute_loss_func = self._compute_in_batch_neg_loss
```

3. 工厂 (Factory)

工厂: inference/auto\_embedder.py

工厂 abc 工厂 abc

4. 建造者 (Builder)

工厂: AbsEmbedderRunner AbsEvalRunner

Runner 工厂model数据集trainer 工厂

5.2

1. 分离关注点 (Separation of Concerns)

| 模块         | 功能 |
|------------|----|
| inference  | 推理 |
| finetune   | 微调 |
| evaluation | 评估 |
| abc        | 接口 |

2. 接口

- 接口: 工厂 AbsEmbedder / AbsReranker 工厂 encode\_single\_device / compute\_score\_single\_gpu
- 接口: 工厂 AbsEmbedderModel 工厂
- 接口: 工厂 AbsEvalDataLoader 工厂

3. 工厂

- 工厂
- 工厂 \*\*kwargs 工厂

- `train.py`

4. `train.py`

1. `train.py`: `train.py` GPU
2. `train.py`: `train.py` GPU `train.py`
3. `train.py`:
  - `train.py` ( `sub_batch_size` )
  - `train.py` ( `share_memory` )
  - `train.py` ( `corpus_embd_save_dir` )
4. `train.py`: FP16/BF16 `train.py`

5. `train.py`

| <code>train.py</code>            | <code>train.py</code>                             |
|----------------------------------|---------------------------------------------------|
| <code>train.py</code>            | In-batch negatives                                |
| <code>train.py</code>            | Cross-device negatives ( <code>train.py</code> )  |
| <code>train.py</code>            | Knowledge Distillation                            |
| Matryoshka <code>train.py</code> | MRL, <code>train.py</code>                        |
| <code>train.py</code> Batch      | <code>train.py</code> batch <code>train.py</code> |
| <code>train.py</code>            | <code>train.py</code>                             |

## 5.3 □□□□

```
□
AbsEmbedder (ABC)
├─ encode_queries()
├─ encode_corpus()
├─ encode()
└─ encode_single_device() [abstract]

AbsReranker (ABC)
├─ compute_score()
└─ compute_score_single_gpu() [abstract]

□(Embedder):
AbsEmbedderModel (ABC, nn.Module)
├─ encode() [abstract]
├─ compute_loss() [abstract]
├─ compute_score() [abstract]
├─ save() [abstract]
├─ forward()
├─ _compute_in_batch_neg_loss()
├─ _compute_cross_device_neg_loss()
└─ distill_loss()

AbsEmbedderTrainDataset (Dataset)
AbsEmbedderCollator (DataCollatorWithPadding)
AbsEmbedderTrainer (Trainer)
AbsEmbedderRunner (ABC)

□(Reranker):
[□Embedder]

□
AbsEvalRunner
├─ get_models()
└─ run()

AbsEvaluator
├─ __call__()
└─ evaluate_results()

AbsEvalDataLoader (ABC)
├─ load_corpus()
├─ load_queries()
└─ load_qrels()

EvalRetriever (ABC)
└─ EvalDenseRetriever

EvalReranker
```

---

FlagEmbedding [ ]

- 1. [ ]: [ ]
- 2. [ ]: [ ]
- 3. [ ]: [ GPU [ ]
- 4. [ ]: [MRL][KD][ ]

[ ] FlagEmbedding [ ]

# 03 inference module

## FlagEmbedding [ ]

[ ]

### Diagram 2

Mermaid Diagram

[ ]

### Diagram 3

Mermaid Diagram



目录

| 名称                   | 类型           | 模型            | 输出                 |
|----------------------|--------------|---------------|--------------------|
| FlagAutoModel        | 模型           | Embedder      | Embedding          |
| FlagAutoReranker     | 模型           | Reranker      | Score              |
| BaseEmbedder         | Encoder-only | BERT          | CLS/Mean Pooling   |
| M3Embedder           | Encoder-only | BGE-M3        | Embedding          |
| BaseLLMEmbedder      | Decoder-only | LLM           | Last Token Pooling |
| ICLLMEmbedder        | Decoder-only | Few-shot      | Embedding          |
| BaseReranker         | Encoder-only | Cross-Encoder | Score              |
| LayerWiseLLMReranker | Decoder-only | LLM           | Score              |

FlagEmbedding 推理 Embedder Reranker

目录

- 1. 目录
- 2. 目录
- 3. Embedder
- 4. Reranker

1. 目录

/workspace/FlagEmbedding/inference/



```

class FlagAutoModel:
    @classmethod
    def from_finetuned(
        cls,
        model_name_or_path: str,
        model_class: Optional[Union[str, EmbedderModelClass]] = None,
        normalize_embeddings: bool = True,
        use_fp16: bool = True,
        use_bf16: bool = False,
        query_instruction_for_retrieval: Optional[str] = None,
        devices: Optional[Union[str, List[str]]] = None,
        pooling_method: Optional[str] = None,
        trust_remote_code: Optional[bool] = None,
        query_instruction_format: Optional[str] = None,
        truncate_dim: Optional[int] = None,
        **kwargs,
    ):
        # ...
        # ...
        # Embedder

```

□ □ □ □ □

1. `checkpoint`
2. `model_mapping`
3. `pooling_method, trust_remote_code, query_instruction_format`
4. `Embedder`

```
from FlagEmbedding import FlagAutoModel

# BGE-M3
model = FlagAutoModel.from_finetuned("BAAI/bge-m3")
```

## 2.2 FlagAutoReranker - Reranker

```

/workspace/FlagEmbedding/inference/auto_reranker.py

```

FlagAutoReranker [ ] FlagAutoModel [ ] Reranker [ ]

```
class FlagAutoReranker:
    @classmethod
    def from_finetuned(
        cls,
        model_name_or_path: str,
        model_class: Optional[Union[str, RerankerModelClass]] = None,
        use_fp16: bool = False,
        trust_remote_code: Optional[bool] = None,
        **kwargs,
    ):
        # ..
```

## 2.3 Embedder

workspace/FlagEmbedding/inference/embedder/model\_mapping.py

### 2.3.1 EmbedderModelClass

```
class EmbedderModelClass(Enum):
    ENCODER_ONLY_BASE = "encoder-only-base"
    ENCODER_ONLY_M3 = "encoder-only-m3"
    DECODER_ONLY_BASE = "decoder-only-base"
    DECODER_ONLY_ICL = "decoder-only-icl"
    DECODER_ONLY_PSEUDO_MOE = "decoder-only-pseudo_moe"
```

### 2.3.2 EmbedderConfig

```
@dataclass
class EmbedderConfig:
    model_class: Type[AbsEmbedder]
    pooling_method: PoolingMethod
    trust_remote_code: bool = False
    query_instruction_format: str = "{}{}"
```

### 2.3.3 PoolingMethod

```
class PoolingMethod(Enum):
    LAST_TOKEN = "last_token"
    CLS = "cls"
    MEAN = "mean"
```

### 2.3.4

AUTO\_EMBEDDER\_MAPPING

| Embedding       | Model                                                                                 |
|-----------------|---------------------------------------------------------------------------------------|
| BGE             | bge-m3 , bge-large-en-v1.5 , bge-base-en-v1.5 , bge-small-en-v1.5 , bge-large-zh-v1.5 |
| Qwen3-Embedding | Qwen3-Embedding-0.6B , Qwen3-Embedding-4B , Qwen3-Embedding-8B                        |
| E5              | e5-mistral-7b-instruct , e5-large-v2 , multilingual-e5-large                          |
| GTE             | gte-Qwen2-7B-instruct , gte-large-en-v1.5 , gte-large-zh                              |
| SFR             | SFR-Embedding-2_R , SFR-Embedding-Mistral                                             |

## 2.4 Reranker

File: /workspace/FlagEmbedding/inference/reranker/model\_mapping.py

### 2.4.1 RerankerModelClass

```
class RerankerModelClass(Enum):
    ENCODER_ONLY_BASE = "encoder-only-base"
    DECODER_ONLY_BASE = "decoder-only-base"
    DECODER_ONLY_LAYERWISE = "decoder-only-layerwise"
    DECODER_ONLY_LIGHTWEIGHT = "decoder-only-lightweight"
```

### 2.4.2 RerankerConfig

```
@dataclass
class RerankerConfig:
    model_class: Type[AbsReranker]
    trust_remote_code: bool = False
```

### 2.4.3 Reranker

| Model                                | Architecture             |
|--------------------------------------|--------------------------|
| bge-reranker-base                    | Encoder-only             |
| bge-reranker-large                   | Encoder-only             |
| bge-reranker-v2-m3                   | Encoder-only             |
| bge-reranker-v2-gemma                | Encoder-only             |
| bge-reranker-v2-minicpm-layerwise    | Encoder-only layerwise   |
| bge-reranker-v2.5-gemma2-lightweight | Encoder-only lightweight |

---

## 3. Embedder Embedder

---

### 3.1 Encoder-only Embedder - BaseEmbedder

Path /workspace/FlagEmbedding/inference/embedder/encoder\_only/base.py

**BaseEmbedder** Encoder-only Embedder Interface AbsEmbedder

#### 3.1.1 Init

```
class BaseEmbedder(AbsEmbedder):
    DEFAULT_POOLING_METHOD = "cls"

    def __init__(
        self,
        model_name_or_path: str,
        normalize_embeddings: bool = True,
        use_fp16: bool = True,
        use_bf16: bool = False,
        query_instruction_for_retrieval: Optional[str] = None,
        query_instruction_format: str = "{}{}",
        devices: Optional[Union[str, List[str]]] = None,
        pooling_method: str = "cls",
        trust_remote_code: bool = False,
        cache_dir: Optional[str] = None,
        batch_size: int = 256,
        query_max_length: int = 512,
        passage_max_length: int = 512,
        convert_to_numpy: bool = True,
        truncate_dim: Optional[int] = None,
        **kwargs: Any,
    ):
        super().__init__(...)
        self.pooling_method = pooling_method

        # tokenizer model
        self.tokenizer = AutoTokenizer.from_pretrained(
            model_name_or_path,
            trust_remote_code=trust_remote_code,
            cache_dir=cache_dir
        )
        self.model = AutoModel.from_pretrained(
            model_name_or_path,
            trust_remote_code=trust_remote_code,
            cache_dir=cache_dir,
            dtype=self.get_model_torch_dtype(),
        )
```

Options

- Options FP32, FP16, BF16

- pooling over CLS
- tokenizer

### 3.1.2 `encode_single_device`

```

@torch.no_grad()
def encode_single_device(
    self,
    sentences: Union[List[str], str],
    batch_size: int = 256,
    max_length: int = 512,
    convert_to_numpy: bool = True,
    device: Optional[str] = None,
    **kwargs: Any
):
    # 1. []
    if device is None:
        device = self.target_devices[0]
    if device == "cpu":
        self.model.float()
    self.model.to(device)
    self.model.eval()

    # 2. []
    all_inputs = []
    for start_index in trange(0, len(sentences), batch_size, desc='pre tokenize'):
        sentences_batch = sentences[start_index:start_index + batch_size]
        inputs_batch = self.tokenizer(
            sentences_batch,
            truncation=True,
            max_length=max_length,
            **kwargs
        )
        # []
        inputs_batch = [{
            k: inputs_batch[k][i] for k in inputs_batch.keys()
        } for i in range(len(sentences_batch))]
        all_inputs.extend(inputs_batch)

    # 3. []padding[]
    length_sorted_idx = np.argsort([-len(x['input_ids']) for x in all_inputs])
    all_inputs_sorted = [all_inputs[i] for i in length_sorted_idx]

    # 4. []batch []OOM[]
    flag = False
    while flag is False:
        try:
            inputs_batch = self.tokenizer.pad(
                all_inputs_sorted[: batch_size],
                padding=True,
                return_tensors='pt',
                **kwargs
            ).to(device)
            last_hidden_state = self.model(**inputs_batch, return_dict=True).last_hidden_state
            embeddings = self.pooling(last_hidden_state, inputs_batch['attention_mask'])
            flag = True
        except RuntimeError as e:
            batch_size = batch_size * 3 // 4 # []75%
        except torch.cuda.OutOfMemoryError as e:
            batch_size = batch_size * 3 // 4

    # 5. []
    all_embeddings = []

```



```

for start_index in tqdm(range(0, len(sentences), batch_size), desc="Inference Embeddings"):
    inputs_batch = all_inputs_sorted[start_index:start_index + batch_size]
    inputs_batch = self.tokenizer.pad(
        inputs_batch,
        padding=True,
        return_tensors='pt',
        **kwargs
    ).to(device)

    last_hidden_state = self.model(**inputs_batch, return_dict=True).last_hidden_state
    embeddings = self.pooling(last_hidden_state, inputs_batch['attention_mask'])
    embeddings = self._truncate_embeddings(embeddings)

    if self.normalize_embeddings:
        embeddings = torch.nn.functional.normalize(embeddings, dim=-1)

    embeddings = cast(torch.Tensor, embeddings)
    if convert_to_numpy:
        embeddings = self._convert_to_numpy(embeddings, device=device)
    all_embeddings.append(embeddings)

# 6. []
if convert_to_numpy:
    all_embeddings = np.concatenate(all_embeddings, axis=0)
else:
    all_embeddings = torch.cat(all_embeddings, dim=0)

all_embeddings = all_embeddings[np.argsort(length_sorted_idx)]

# 7. []
if input_was_string:
    return all_embeddings[0]
return all_embeddings

```

□□□□□□

1. □ **tokenization** □□□□ padding □□□□□□□□□□
2. □□□□ □□□□□□□□□□ batch □□□□□□□□□□ padding
3. □□□ **batch size** □□□ OOM □□□□□ batch □□□75%□
4. □□□□□□ L2 □□□
5. □□□□□□□ truncate\_dim □□□□□□

### 3.1.3 Pooling

```
def pooling(
    self,
    last_hidden_state: torch.Tensor,
    attention_mask: Optional[torch.Tensor] = None
):
    if self.pooling_method == 'cls':
        return last_hidden_state[:, 0] # <[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]> token
    elif self.pooling_method == 'mean':
        # pooling attention mask
        s = torch.sum(last_hidden_state * attention_mask.unsqueeze(-1).float(), dim=-1)
        d = attention_mask.sum(dim=-1, keepdim=True).float()
        return s / d
    else:
        raise NotImplementedError(f"pooling method {self.pooling_method} not implemented")
```

#### pooling

- **cls** - <[BOS\_never\_used\_51bce0c785ca2f68081bfa7d91973934]> token
- **mean** - token

## 3.2 BGE-M3 - M3Embedder

/workspace/FlagEmbedding/inference/embedder/encoder\_only/m3.py

M3Embedder BGE-M3 embedding

### 3.2.1

BGE-M3

- **Dense Embedding** -
- **Sparse Embedding** -
- **CoBERT** - token embedding

### 3.2.2 encode\_single\_device

```

@torch.no_grad()
def encode_single_device(
    self,
    sentences: Union[List[str], str],
    batch_size: int = 256,
    max_length: int = 512,
    return_dense: bool = True,
    return_sparse: bool = False,
    return_colbert_vecs: bool = False,
    device: Optional[str] = None,
    **kwargs: Any
):
    # ... BaseEmbedder ...

    # []
    all_dense_embeddings, all_lexical_weights, all_colbert_vecs = [], [], []
    for start_index in tqdm(range(0, len(sentences), batch_size), desc="Inference Embeddings"):
        inputs_batch = all_inputs_sorted[start_index:start_index + batch_size]
        inputs_batch = self.tokenizer.pad(
            inputs_batch,
            padding=True,
            return_tensors='pt',
            **kwargs
        ).to(device)

        outputs = self.model(
            inputs_batch,
            return_dense=return_dense,
            return_sparse=return_sparse,
            return_colbert_vecs=return_colbert_vecs,
            truncate_dim=self.truncate_dim
        )

        # []dense embedding
        if return_dense:
            all_dense_embeddings.append(
                self._convert_to_numpy(outputs['dense_vecs'], device=device)
            )

        # []sparse embedding[]lexical weights[]
        if return_sparse:
            token_weights = outputs['sparse_vecs'].squeeze(-1)
            all_lexical_weights.extend(
                list(map(
                    _process_token_weights,
                    self._convert_to_numpy(token_weights, device=device),
                    self._convert_to_numpy(inputs_batch['input_ids'], device=device).tolist()
                )))

        # []colbert []
        if return_colbert_vecs:
            all_colbert_vecs.extend(
                list(map(
                    _process_colbert_vecs,
                    self._convert_to_numpy(outputs['colbert_vecs'], device=device),
                    self._convert_to_numpy(inputs_batch['attention_mask'], device=device)
                )))

```

```

    )

    # ..
    return {
        "dense_vecs": all_dense_embeddings,
        "lexical_weights": all_lexical_weights,
        "colbert_vecs": all_colbert_vecs
    }

```

~~~~~

1. `_process_token_weights` - ~~~~~

- ~~~~ token<[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]>[SEP][PAD][UNK]
- ~ token id ~~~~ token
- ~~~~~

2. `_process_colbert_vecs` - ~ ColBERT ~

- ~ padding ~~~~
- ~~~<[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]> ~ embedding

3.2.3 ~~~~~ - `compute_score`

BGE-M3 ~~~~~

```

def compute_score(
    self,
    sentence_pairs: Union[List[Tuple[str, str]], Tuple[str, str]],
    batch_size: Optional[int] = None,
    max_query_length: Optional[int] = None,
    max_passage_length: Optional[int] = None,
    weights_for_different_modes: Optional[List[float]] = None,
    **kwargs: Any
) -> Dict[
    Literal["colbert", "sparse", "dense", "sparse+dense", "colbert+sparse+dense"],
    List[float]
]:
    # ~
    if len(self.target_devices) == 1:
        return self.compute_score_single_device(...)
    else:
        return self.compute_score_multi_process(...)

```

`compute_score_single_device`~

```

@torch.no_grad()
def compute_score_single_device(
    self,
    sentence_pairs: Union[List[Tuple[str, str]], Tuple[str, str]],
    batch_size: int = 256,
    max_query_length: int = 512,
    max_passage_length: int = 512,
    weights_for_different_modes: Optional[List[float]] = None,
    device: Optional[str] = None,
    **kwargs: Any
):
    # 1. □
    queries_inputs = _tokenize([pair[0] for pair in sentences_batch], max_query_length)
    passages_inputs = _tokenize([pair[1] for pair in sentences_batch], max_passage_length)

    # 2. □
    queries_output = self.model(
        queries_inputs,
        return_dense=True, return_sparse=True, return_colbert_vecs=True,
        return_sparse_embedding=True
    )
    passages_output = self.model(
        passages_inputs,
        return_dense=True, return_sparse=True, return_colbert_vecs=True,
        return_sparse_embedding=True
    )

    # 3. □
    dense_scores = self.model.compute_dense_score(q_dense, p_dense)
    sparse_scores = self.model.compute_sparse_score(q_sparse, p_sparse)
    colbert_scores = self.model.compute_colbert_score(q_colbert, p_colbert, q_mask)

    # 4. □
    if weights_for_different_modes is None:
        weights_for_different_modes = [1., 1., 1.] # [dense, sparse, colbert]
        weight_sum = 3

    # 5. □
    return {
        'colbert': colbert_scores,
        'sparse': sparse_scores,
        'dense': dense_scores,
        'sparse+dense': (sparse * w1 + dense * w0) / (w0 + w1),
        'colbert+sparse+dense': (colbert * w2 + sparse * w1 + dense * w0) / sum(weights)
    }

```

3.3 □□□-only □□□□ - BaseLLMEmbedder

□□□□ /workspace/FlagEmbedding/inference/embedder/decoder_only/base.py

BaseLLMEmbedder □□□□-only □□□□□□□□ last token pooling□

3.3.1 Last Token Pooling

```
def last_token_pool(last_hidden_states: torch.Tensor,
                    attention_mask: torch.Tensor) -> torch.Tensor:
    # padding
    left_padding = (attention_mask[:, -1].sum() == attention_mask.shape[0])
    if left_padding:
        return last_hidden_states[:, -1] # padding token
    else:
        # padding token
        sequence_lengths = attention_mask.sum(dim=1) - 1
        batch_size = last_hidden_states.shape[0]
        return last_hidden_states[
            torch.arange(batch_size, device=last_hidden_states.device),
            sequence_lengths
        ]
```

####

- #####/ padding
- ##### token #####

3.4 ICL - ICLLLMEmbedder

/workspace/FlagEmbedding/inference/embedder/decoder_only/icl.py

ICLLMEmbedder Few-Shot #####

3.4.1

```
def set_examples(self, examples_for_task: Optional[List[dict]] = None):
    """Few-Shot """
    if examples_for_task is None and self.examples_for_task is None:
        self.prefix = ''
    else:
        # 
        eg_paris = []
        for example in examples_for_task or self.examples_for_task:
            eg_paris.append(
                self.get_detailed_example(
                    self.examples_instruction_format,
                    example.get('instruct', self.query_instruction_for_retrieval),
                    example.get('query', ''),
                    example.get('response', '')
                )
            )
        self.prefix = '\n\n'.join(eg_paris) + '\n\n'
```

3.4.2 `encode_queries`

```
@torch.no_grad()
def encode_queries_single_device(
    self,
    queries: Union[List[str], str],
    batch_size: int = 256,
    max_length: int = 512,
    convert_to_numpy: bool = True,
    device: Optional[str] = None,
    **kwargs: Any
):
    # 1. input_texts
    if self.query_instruction_for_retrieval is not None:
        input_texts = [
            self.get_detailed_instruct(
                self.query_instruction_format,
                self.query_instruction_for_retrieval,
                query
            ) for query in queries
        ]

    # 2. prefix_ids + suffix_ids
    prefix_ids = self.tokenizer(self.prefix, add_special_tokens=False)['input_ids']
    suffix_ids = self.tokenizer(self.suffix, add_special_tokens=False)['input_ids']

    # 3. new_max_length
    new_max_length = (len(prefix_ids) + len(suffix_ids) + max_length + 8) // 8 * 8 + 8

    # 4. BaseLLMEmbedder..
```

`input_texts` `[0 + [0 + [0`

3.5 `MoE` - `PseudoMoELLMEmbedder`

`/workspace/FlagEmbedding/inference/embedder/decoder_only/pseudo_moe.py`

`PseudoMoELLMEmbedder` `MoE` `Mixture of Experts`

3.5.1 实现

```
def _resolve_domain(self, kwargs: Any) -> Optional[str]:
    """kwargs"""
    domain = kwargs.pop("domain_for_pseudo_moe", None)
    if domain is None:
        domain = kwargs.pop("domain", None)
    if domain is None:
        domain = self.domain_for_pseudo_moe
    return domain

@torch.no_grad()
def encode_single_device(
    self,
    sentences: Union[List[str], str],
    batch_size: int = 256,
    max_length: int = 512,
    convert_to_numpy: bool = True,
    device: Optional[str] = None,
    **kwargs: Any
):
    #
    domain = self._resolve_domain(kwargs)

    #
    if domain is not None and hasattr(self.model, "set_domain"):
        self.model.set_domain(domain)

    # forward
    model_forward_kwargs = {"return_dict": True}
    if domain is not None:
        model_forward_kwargs["domain"] = domain

    # domain
    last_hidden_state = self.model(*inputs_batch, **model_forward_kwargs).last_hidden_state
    # ...
```

4. Reranker 实现

4.1 仅-encoder - BaseReranker

实现 /workspace/FlagEmbedding/inference/reranker/encoder_only/base.py

BaseReranker 仅 Reranker

4.1.1 实验 - compute_score_single_gpu

```

@torch.no_grad()
def compute_score_single_gpu(
    self,
    sentence_pairs: Union[List[Tuple[str, str]], Tuple[str, str]],
    batch_size: Optional[int] = None,
    query_max_length: Optional[int] = None,
    max_length: Optional[int] = None,
    normalize: Optional[bool] = None,
    device: Optional[str] = None,
    **kwargs: Any
) -> List[float]:
    # 1. []
    all_inputs = []
    for start_index in trange(0, len(sentence_pairs), batch_size, desc="pre tokenize"):
        sentences_batch = sentence_pairs[start_index:start_index + batch_size]
        queries = [s[0] for s in sentences_batch]
        passages = [s[1] for s in sentences_batch]

        # []tokenize []
        queries_inputs_batch = self.tokenizer(
            queries,
            return_tensors=None,
            add_special_tokens=False,
            max_length=query_max_length,
            truncation=True,
            **kwargs
        )['input_ids']
        passages_inputs_batch = self.tokenizer(
            passages,
            return_tensors=None,
            add_special_tokens=False,
            max_length=max_length,
            truncation=True,
            **kwargs
        )['input_ids']

        # [BOS_never_used_51bce0c785ca2f68081bfa7d91973934]> + query + SEP + passage + SEP
        for q_inp, d_inp in zip(queries_inputs_batch, passages_inputs_batch):
            item = self.tokenizer.prepare_for_model(
                q_inp,
                d_inp,
                truncation='only_second', # []
                max_length=max_length,
                padding=False,
            )
            all_inputs.append(item)

    # 2. []
    length_sorted_idx = np.argsort([-len(x['input_ids']) for x in all_inputs])
    all_inputs_sorted = [all_inputs[i] for i in length_sorted_idx]

    # 3. []batch []
    flag = False
    while flag is False:
        try:
            test_inputs_batch = self.tokenizer.pad(all_inputs_sorted[:min(...)]).to(device)
            scores = self.model(**test_inputs_batch, return_dict=True).logits.view(-1, ).float()
            flag = True

```

```

except (RuntimeError, torch.cuda.OutOfMemoryError):
    batch_size = batch_size * 3 // 4

# 4. []
all_scores = []
for start_index in tqdm(range(0, len(all_inputs_sorted), batch_size), desc="Compute Scores"):
    inputs = self.tokenizer.pad(all_inputs_sorted[start_index:start_index + batch_size]).to(device)
    scores = self.model(*inputs, return_dict=True).logits.view(-1, ).float()
    all_scores.extend(scores.cpu().numpy().tolist())

# 5. []
all_scores = [all_scores[idx] for idx in np.argsort(length_sorted_idx)]

# 6. Sigmoid []
if normalize:
    all_scores = [sigmoid(score) for score in all_scores]

return all_scores

```

[[[[[<[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]> [[SEP] [[SEP]

4.2 Decoder-only Reranker - BaseLLMReranker

[[[[[/workspace/FlagEmbedding/inference/reranker/decoder_only/base.py

BaseLLMReranker [[LLM [[Reranker[[[[["Yes" [["No" [[[[[[[[[[

4.2.1 [[[[[

```

# []
if prompt is None:
    prompt = "Given a query A and a passage B, determine whether the passage contains an answer to the

# []
item['input_ids'] = [bos_token] + query_inputs + sep + passage_inputs + sep + prompt_inputs

```

[[[[[BOS] [[SEP] [[SEP] []

4.2.2 Last Logit Pooling

```

def last_logit_pool(logits: Tensor,
                    attention_mask: Tensor) -> Tensor:
    left_padding = (attention_mask[:, -1].sum() == attention_mask.shape[0])
    if left_padding:
        return logits[:, -1] # []padding[]token
    else:
        sequence_lengths = attention_mask.sum(dim=1) - 1
        batch_size = logits.shape[0]
        return torch.stack([
            logits[i, sequence_lengths[i], :]
            for i in range(batch_size)
        ], dim=0)

```

token "Yes" logit

```
self.yes_loc = self.tokenizer('Yes', add_special_tokens=False)['input_ids'][0]
scores = last_logit_pool(logits, inputs['attention_mask'])
scores = scores[:, self.yes_loc] # 'Yes'
```

4.3 -wise - LayerWiseLLMReranker

/workspace/FlagEmbedding/inference/reranker/decoder_only/layerwise.py

LayerWiseLLMReranker

4.3.1

```
def compute_score_single_gpu(
    self,
    sentence_pairs: ...,
    cutoff_layers: Optional[List[int]] = None, # 
    ...
):
    # cutoff_layers
    outputs = self.model(
        **batch_inputs,
        output_hidden_states=True,
        cutoff_layers=cutoff_layers # 
    )

    # logits
    all_logits = outputs.logits # List[Tensor]
    tmp_all_scores = []
    for logits in all_logits:
        scores = last_logit_pool_layerwise(logits, inputs['attention_mask'])
        tmp_all_scores.append(scores.contiguous())

    # 
    all_scores = []
    for i in range(len(tmp_all_scores)):
        all_scores.append(tmp_all_scores[i].cpu().float().tolist())

    # 
    for i in range(len(all_scores)):
        all_scores[i] = [all_scores[i][idx] for idx in np.argsort(length_sorted_idx)]
        if normalize:
            all_scores[i] = [sigmoid(score) for score in all_scores[i]]

    return all_scores
```

4.4 - LightweightLLMReranker

/workspace/FlagEmbedding/inference/reranker/decoder_only/lightweight.py

5. 实现方案

5.1 实现

问题	原因	实现
Padding	OOM	BaseEmbedder.encode_single_device
Batch Size	OOM	BaseEmbedder.encode_single_device
Last Token Pooling	Decoder-only	BaseLLMEmbedder
Embedder	BGE-M3	M3Embedder
Reranker	LLM Reranker	BaseLLMReranker

5.2 实现

- [auto_embedder.py](#)
 - [auto_reranker.py](#)
 - [embedder/model_mapping.py](#)
 - [embedder/encoder_only/base.py](#)
 - [embedder/encoder_only/m3.py](#)
 - [embedder/decoder_only/base.py](#)
 - [reranker/encoder_only/base.py](#)
 - [reranker/decoder_only/base.py](#)
 - [reranker/decoder_only/layerwise.py](#)
 - [reranker/decoder_only/lightweight.py](#)
-

04 finetune module

FlagEmbedding 模块

模块

Diagram 4

Mermaid Diagram

模块

Diagram 5

Mermaid Diagram

模块

模块	模块	模块	模块
模块	模块	模块batch模块	AbsEmbedderModel
模块	模块	模块GPU模块	AbsEmbedderModel
模块	模块	模块→模块	distill_loss()
MRL	模块	模块	forward()
模块	模块	模块→模块	EncoderOnlyEmbedderM3Model

模块

- 模块
- 模块
- Embedder 模块

- [Reranker](#)
-
-
-

FlagEmbedding Embedder Reranker

```
finetune/
├── embedder/
│   ├── encoder_only/
│   │   ├── base/      # Encoder-Only
│   │   └── m3/        # BGE-M3
│   └── decoder_only/
│       ├── base/      # Decoder-Only
│       └── icl/       # In-Context Learning
└── reranker/
    ├── encoder_only/
    │   ├── base/      # Encoder-Only Reranker
    └── decoder_only/
        ├── base/      # Decoder-Only Reranker
        └── layerwise/ #
```

AbsEmbedderModel

📄: [/workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py](#)

AbsEmbedderModel Embedder Embedder

1. `encode(features)` -
2. `compute_loss(scores, target)` -
3. `compute_score(q_reps, p_reps)` - Query-Passage
4. `forward(queries, passages, teacher_scores, no_in_batch_neg_flag)` -

□□□□□□

```
# 1. □queries □passages
q_reps = self.encode(queries)
p_reps = self.encode(passages)

# 2. □
if no_in_batch_neg_flag:
    compute_loss_func = self._compute_no_in_batch_neg_loss
else:
    if self.negatives_cross_device:
        compute_loss_func = self._compute_cross_device_neg_loss
    else:
        compute_loss_func = self._compute_in_batch_neg_loss

# 3. □
scores, loss = compute_loss_func(q_reps, p_reps, teacher_targets=teacher_targets)
```

AbsRerankerModel

□□□□: </workspace/FlagEmbedding/abc/finetune/reranker/AbsModeling.py>

AbsRerankerModel □ Reranker □□□□□□□□□□□□□□□□

□□□□

- □□□□□□□□
- □□□□□□
- □□□ encode □□□□□□-□□□□□□□□

Embedder □□□□

1. Encoder-Only Base □□

□□□□: /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/modeling.py

□□□: **BiEncoderOnlyEmbedderModel**

```
class BiEncoderOnlyEmbedderModel(AbsEmbedderModel):
    def __init__(
        self,
        base_model: PreTrainedModel,
        tokenizer: PreTrainedTokenizer = None,
        negatives_cross_device: bool = False,
        temperature: float = 1.0,
        sub_batch_size: int = -1,
        kd_loss_type: str = 'kl_div',
        use_mrl: bool = False,
        mrl_dims: List[int] = [],
        sentence_pooling_method: str = 'cls',
        normalize_embeddings: bool = False,
    )
```

□□□□

1. **_sentence_embedding(last_hidden_state, attention_mask)**
 - □□□□□□□: **cls**, **mean**, **last_token**
 - **cls**: □ <[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]> token □□
 - **mean**: □□□ token □□□□
 - **last_token**: □□□□□□□□ token
2. **encode(features)**
 - □□□□□□□□□ OOM
 - □□ MRL (Matryoshka Representation Learning)
 - □□ embedding □□□
3. **compute_score(q_reps, p_reps)**
 - □□□□□□□□
 - □□□□□□□□□□□□

2. Encoder-Only M3 □□

□□□□: /workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py

□□□: **EncoderOnlyEmbedderM3Model**

BGE-M3 □□□□□□□□□□□□

- **Dense Embedding**: □□□□□□
- **Sparse Embedding**: □□□□□□□□□□
- **CoBERT**: □□□ token □□□

□□□□

1. □□□□□□□
 - **_dense_embedding()**: □□□□□□□□□□

- `_colbert_embedding()`: token

2.

```
python
```


--	--	--	--	--	--	--	--

```
dense_scores, dense_loss = compute_loss_func(...)
```

```
sparse_scores, sparse_loss = compute_loss_func(...)
```

```
colbert_scores, colbert_loss = compute_loss_func(...)
```


--	--	--	--

```
ensemble_scores, ensemble_loss = compute_loss_func(...)
```


--	--	--	--

```
loss = (loss + ensemble_loss + 0.1 * sparse_loss + Colbert_loss) / 4
```

...

1. 自蒸馏 (Self-Distillation)

-
- | | | | | | | | | |
|--|--|--|--|--|--|--|--|--|
| | | | | | | | | |
|--|--|--|--|--|--|--|--|--|

3. Decoder-Only Base ☐

```

□□□□: /workspace/FlagEmbedding/finetune/embedder/decoder_only/base/modeling.py

```

□□□: BiDecoderOnlyEmbedderModel

-  last_token 

4. Decoder-Only ICL ☐

```

[ ] [ ] [ ] [ ]: /workspace/FlagEmbedding/finetune/embedder/decoder_only/icl/modeling.py

```

```

[ ] [ ] [ ]: BiDecoderOnlyEmbedderICLModel

```

- [illegible]

Reranker 📄📄📄

1. Encoder-Only Base 📄

📄📄📄: /workspace/FlagEmbedding/finetune/reranker/encoder_only/base/modeling.py

📄📄📄: **CrossEncoderModel**

```
class CrossEncoderModel(AbsRerankerModel):
    def encode(self, features):
        return self.model(**features, return_dict=True).logits
```

- 📄📄 Cross-Encoder 📄
- 📄📄📄📄📄📄📄📄
- 📄📄📄📄 logits 📄📄

2. Decoder-Only Base 📄

📄📄📄: /workspace/FlagEmbedding/finetune/reranker/decoder_only/base/modeling.py

📄📄📄: **CrossDecoderModel**

```
def encode(self, features):
    outputs = self.model(input_ids=features['input_ids'],
                        attention_mask=features['attention_mask'],
                        position_ids=features['position_ids'] if 'position_ids' in features.keys() else None,
                        output_hidden_states=True)
    scores = outputs.logits[:, -1, self.yes_loc]
    return scores.contiguous()
```

- 📄 Decoder 📄📄 Reranker
- 📄 "Yes" token 📄📄📄📄📄

3. Decoder-Only Layerwise 📄

📄📄📄: /workspace/FlagEmbedding/finetune/reranker/decoder_only/layerwise/modeling.py

📄📄📄: **CrossDecoderModel (Layerwise 📄)**

- 📄📄📄
- 📄📄📄📄📄
- 📄📄📄📄📄📄📄

```
def forward(self, pair, teacher_scores=None):
    ranker_logits = self.encode(pair)

    if self.training:
        loss = 0
        # 1
        for logits in ranker_logits:
            loss += self.compute_loss(...)

        # 2 3
        if teacher_scores is None:
            teacher_scores = ranker_logits[-1]
            teacher_targets = torch.softmax(teacher_scores.detach(), dim=-1)
            for logits in ranker_logits[:-1]:
                loss += distillation_loss
```

□□□□□□

1. □□□□□□

□□□□□□

AbsEmbedderModel □□□□□□□□□□

□□	□□	□□
□□□□	<code>_compute_no_in_batch_neg_loss</code>	□□□□□□□□□□
□□□□	<code>_compute_in_batch_neg_loss</code>	□□□□ batch □□□□□□□□
□□□□	<code>_compute_cross_device_neg_loss</code>	□□□□□□□□□□

□□□□

```
score(q, p) = q · p / temperature
loss = CrossEntropy(score, target)
```

2. □□□□ (In-Batch Negatives)

□□□□: [AbsEmbedderModel._compute_in_batch_neg_loss](#)

□□□□

- batch □□□□ query□batch □□□□ passage □□□□
- query □□□□□□□□ score □□

3. `target = idx * group_size`

```
def _compute_in_batch_neg_loss(self, q_reps, p_reps, teacher_targets=None, ...):
    # query-passage
    scores = self.compute_score(q_reps, p_reps) # (batch_size, batch_size * group_size)

    # query
    idxs = torch.arange(q_reps.size(0), device=q_reps.device)
    targets = idxs * group_size # (batch_size)

    #
    loss = self.compute_loss(scores, targets)
    return scores, loss
```

3. `Embedder` (Cross-Device Negatives)

`Embedder`: [AbsEmbedderModel.compute_cross_device_neg_loss](#)

`Embedder`

1. `torch.distributed` embedding
2. embedding
3. `Embedder`

```
def _compute_cross_device_neg_loss(self, q_reps, p_reps, ...):
    # embedding
    cross_q_reps = self._dist_gather_tensor(q_reps)
    cross_p_reps = self._dist_gather_tensor(p_reps)

    #
    cross_scores = self.compute_score(cross_q_reps, cross_p_reps)

    #
    cross_idxes = torch.arange(cross_q_reps.size(0))
    cross_targets = cross_idxes * group_size
    loss = self.compute_loss(cross_scores, cross_targets)
    return cross_scores, loss
```

`Embedder`

```
def _dist_gather_tensor(self, t):
    all_tensors = [torch.empty_like(t) for _ in range(self.world_size)]
    dist.all_gather(all_tensors, t)
    all_tensors[self.process_rank] = t #
    all_tensors = torch.cat(all_tensors, dim=0)
    return all_tensors
```

4. `Embedder` (KD)

`Embedder`: [AbsEmbedderModel.distill_loss](#)

📌 KD 📌📌

1. **kl_div**: 📌 KL 📌

```
python loss = - mean(sum(log_softmax(student) * teacher_target))
```

2. **m3_kd_loss**: M3 📌📌📌📌 KD

- 📌📌📌📌📌
- 📌 mask 📌📌📌

```
def distill_loss(kd_loss_type, teacher_targets, student_scores, group_size=None):
    if kd_loss_type == 'kl_div':
        return - torch.mean(
            torch.sum(torch.log_softmax(student_scores, dim=-1) * teacher_targets, dim=-1)
        )
    elif kd_loss_type == 'm3_kd_loss':
        labels = torch.arange(student_scores.size(0)) * group_size
        loss = 0
        mask = torch.zeros_like(student_scores)
        for i in range(group_size):
            temp_target = labels + i
            temp_scores = student_scores + mask
            temp_loss = F.cross_entropy(temp_scores, temp_target, reduction="none")
            loss += torch.mean(teacher_targets[:, i] * temp_loss)
            mask = torch.scatter(mask, dim=-1, index=temp_target.unsqueeze(-1),
                                value=torch.finfo(student_scores.dtype).min)
        return loss
```

5. MRL (Matryoshka Representation Learning)

📌📌📌: [AbsEmbedderModel.forward](#)

📌📌📌

- 📌 embedding 📌📌📌📌📌📌📌📌
- 📌📌📌📌📌📌📌📌📌
- 📌📌📌📌📌📌

```
if self.use_mrl:
    # 📌
    all_loss = torch.tensor(0.0, device=device)
    for dim_q_reps, dim_p_reps in zip(q_reps, p_reps):
        _, mrl_loss = compute_loss_func(dim_q_reps, dim_p_reps, teacher_targets=teacher_targets)
        all_loss += mrl_loss
    loss = all_loss / len(self.mrl_dims)
```


API

API

```
1. __init__(AbsDataset)
   ↓
2. __init__(Collator)
   ↓
3. __init__(encode)
   ↓
4. __init__(compute_score)
   ↓
5. __init__(compute_loss)
   ↓
6. __init__ & __call__
```

API

API: </workspace/FlagEmbedding/abc/finetune/embedder/AbsDataset.py>

API

1. **AbsEmbedderTrainDataset** : `__init__`
 - `json/jsonl` `__init__`
 - `__init__`
 - `__init__`
2. **AbsEmbedderSameDatasetTrainDataset** : `__init__`
 - `__init__` batch `__init__`
 - `__init__` batch size
 - `__init__` `no_in_batch_neg` `__init__`
3. **AbsEmbedderCollator** : `__init__`
 - tokenization
 - padding
 - `__init__`

API

```
{
  "query": "[]",
  "pos": ["[]."],
  "neg": ["[].", "[]."],
  "pos_scores": [0.95], // __KD
  "neg_scores": [0.1, 0.05] // __KD
}
```

□□□□□

□□□□: /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/trainer.py

□□□: **EncoderOnlyEmbedderTrainer**

□□□ **AbsEmbedderTrainer** □□□□□ **_save** □□□

```
class EncoderOnlyEmbedderTrainer(AbsEmbedderTrainer):
    def _save(self, output_dir: Optional[str] = None, state_dict=None):
        # □
        self.model.save(output_dir)
        # □tokenizer
        self.tokenizer.save_pretrained(output_dir)
        # □
        torch.save(self.args, os.path.join(output_dir, "training_args.bin"))
```

□□□□□□□

EmbedderOutput

```
@dataclass
class EmbedderOutput(ModelOutput):
    q_reps: Optional[Tensor] = None      # Query □
    p_reps: Optional[Tensor] = None      # Passage □
    loss: Optional[Tensor] = None        # □
    scores: Optional[Tensor] = None      # □
```

RerankerOutput

```
@dataclass
class RerankerOutput(ModelOutput):
    loss: Optional[Tensor] = None        # □
    scores: Optional[Tensor] = None      # □
```

□□

FlagEmbedding □□□□□□□□□□

1. □□□□□: □□□□□□□□□□
2. □□□□□□□: □□□□□□□□□□

- 3. □□□□□□: MRL□□□□□□□□
- 4. □□□□□□: Encoder-only□□Decoder-only □□
- 5. □□□□□□: BGE-M3 □□□□□□□□□□

05 evaluation module

FlagEmbedding □□□□□□□□

□□□□□□□□

Diagram 6

Mermaid Diagram

□□□□□□□□

Diagram 7

Mermaid Diagram

数据集

名称	来源	语言	规模	用途
MTEB	微软	英语	1	通用
BEIR	微软	英语	15个	通用
MSMARCO	微软	英语	1000	passage/document
MIRACL	微软	18个	1	通用
MKQA	微软	26个	1	通用
MLDR	微软	13个	1	通用
AIR-Bench	微软	100	100	通用
BRIGHT	微软	100	1	short/long

目录

- 1. 介绍
- 2. 数据集
- 3. 数据集
- 3.1 MTEB / C-MTEB
- 3.2 BEIR
- 3.3 MSMARCO
- 3.4 MIRACL
- 3.5 MKQA
- 3.6 MLDR
- 3.7 AIR-Bench
- 3.8 BRIGHT
- 4. 数据集

1. 目录

FlagEmbedding 数据集 + 数据集

→ →

```
FlagEmbedding/
├── abc/evaluation/      #
├── evaluation/
│   ├── air_bench/      # AIR Bench
│   ├── beir/           # BEIR
│   ├── bright/         # BRIGHT
│   ├── custom/         #
│   ├── miracl/         # MIRACL
│   ├── mkqa/           # MKQA
│   ├── mldr/           # MLDR
│   ├── msmarco/        # MSMARCO
│   └── mteb/           # MTEB
```

2. 数据集

2.1 AbsEvalRunner [runner.py](#)

```
class AbsEvalRunner:
    def __init__(self, eval_args, model_args):
        #
        self.retriever, self.reranker = self.load_retriever_and_reranker()
        self.data_loader = self.load_data_loader()
        self.evaluator = self.load_evaluator()
```

- `get_models()` :
- `load_retriever_and_reranker()` :
- `load_data_loader()` :
- `load_evaluator()` :
- `evaluate_metrics()` :
- `run()` :

2.2 AbsEvaluator [evaluator.py](#)

```
class AbsEvaluator:
    def __init__(self, eval_name, data_loader, overwrite=False):
        self.eval_name = eval_name
        self.data_loader = data_loader
        self.overwrite = overwrite
```



1. 
2.  retriever() 
3. 
4. 
5. 

2.3 EvalDenseRetriever & EvalReranker [searcher.py](#)

EvalDenseRetriever

- 
-  Faiss 
- 

EvalReranker

-
- top-k

2.4 AbsEvalDataLoader [data_loader.py](#)

[illegible]

```
class AbsEvalDataLoader(ABC):
    def __init__(self, eval_name, dataset_dir=None, cache_dir=None, ...):
        # []
```

11/11

- `load_corpus()` : 
- `load_queries()` : 
- `load_qrels()` : 
- `_load_remote_*` : 
- `_load_local_*` : 

2.5 AbsEvalArgs & AbsEvalModelArgs [arguments.py](#)

[illegible]

3. 数据集

3.1 MTEB / C-MTEB

数据集 [evaluation/mteb/runner.py](#)

用法

- 数据集 mteb 数据集
- 数据集名称
- 数据集
- 数据集

数据集 MTEBEvalRunner

数据集

1. 数据集 mteb.get_tasks() 数据集
2. 数据集
3. 数据集
4. 数据集

数据集

```
{
  "task_type": {
    "task_name": score,
    "Avg": average_score
  },
  "Avg": overall_average
}
```

3.2 BEIR

数据集

- [evaluation/beir/runner.py](#)
- [evaluation/beir/data_loader.py](#)

用法

- 数据集
- 数据集 15 数据集
- 数据集

数据集

- arguana, climate-fever, cqadupstack, dbpedia-entity
- fever, fiqa, hotpotqa, msmarco, nfcopus, nq
- quora, scidocs, scifact, trec-covid, webis-touche2020

□□□□

- `BEIREvalRunner` : BEIR □□□□
- `BEIREvalDataLoader` : BEIR □□□□

□□□□□

- [□□□□□□□□□□prompts.py](#)
- □□ cqadupstack □□□□

3.3 MSMARCO

□□□□□

- [evaluation/msmarco/runner.py](#)
- [evaluation/msmarco/data_loader.py](#)

□□□

- □□□□□□□□□□
- □□ passage □ document □□□
- □□ dev, dl19, dl20 □□□

□□□□

- `MSMARCOEvalRunner` : MSMARCO □□□□
- `MSMARCOEvalDataLoader` : MSMARCO □□□□

□□□□□

- passage □□ `Tevatron/msmarco-passages`
- document □□ `irds/msmarco-document`

3.4 MIRACL

□□□□□

- [evaluation/miracl/runner.py](#)
- [evaluation/miracl/data_loader.py](#)

□□□

- □□□□□□□□
- □□ 18 □□
- □□ train □ dev □□

□□□□□□

- ar, bn, en, es, fa, fi, fr, hi, id, ja, ko, ru, sw, te, th, zh, de, yo

□□□□

- `MIRACLEvalRunner` : MIRACL □□□□
- `MIRACLEvalDataLoader` : MIRACL □□□□

□□□□□ `miracl/miracl-corpus` (HuggingFace)

3.5 MKQA

📁📁📁

- [evaluation/mkqa/runner.py](#)
- [evaluation/mkqa/data_loader.py](#)

📁📁

- 📁📁📁📁📁📁
- 📁 26 📁📁/📁
- 📁📁📁📁📁📁📁📁

📁📁📁📁

- en, ar, fi, ja, ko, ru, es, sv, he, th, da, de, fr, it, nl, pt, hu, vi, ms, km, no, tr, zh_cn, zh_hk, zh_tw

📁📁📁

- `MKQAEvalRunner` : MKQA 📁📁📁
- `MKQAEvalDataLoader` : MKQA 📁📁📁

📁📁📁

- 📁📁📁📁📁📁📁 BeIR 📁 nq 📁📁
- 📁📁📁📁 [utils/normalize_text.py](#)

📁📁📁 `Shitao/bge-m3-data` (HuggingFace)

3.6 MLDR

📁📁📁

- [evaluation/mldr/runner.py](#)
- [evaluation/mldr/data_loader.py](#)

📁📁

- 📁📁📁📁📁
- 📁 13 📁
- 📁 train, dev, test 📁📁

📁📁📁📁

- ar, de, en, es, fr, hi, it, ja, ko, pt, ru, th, zh

📁📁

- `MLDREvalRunner` : MLDR 📁📁📁
- `MLDREvalDataLoader` : MLDR 📁📁📁

📁📁📁 `Shitao/MLDR` (HuggingFace)

3.7 AIR-Bench

📁📁📁 [evaluation/air_bench/runner.py](#)

□□□

- □□□□ `air-bench` □
- □□□□□□□□□□
- □□□□□□□□□□

□□□□ `AIRBenchEvalRunner`

□□□□□

1. □□ `AIRBench` □□
2. □□□□□□□□□□
3. □□□□

3.8 BRIGHT

□□□□□ [evaluation/bright/runner.py](#)

□□□

- □□□□□□□□ short □ long
- □□□□□□□□□□
- □□□□□□□□□□

□□□□

- `BrightEvalRunner` : BRIGHT □□□□
- `BrightEvalDenseRetriever` : BRIGHT □□□
- `BrightShortEvalDataLoader` / `BrightLongEvalDataLoader` : □□□□

□□□□□

- □□□□□□□□ [prompts.py](#)□
- □□□□□□ `retriever_qrels` □□□□□□□□

4. □□□□□□□□

4.1 □□□□□□□□

1. □□□□ □□□□□□□□□□□□□□
2. □□□□□ □□□□□□□□□□□□□□
3. □□□□□□
 - □□□□□□□□□□
 - □□□□□
 - □□ Faiss □□□□
 - □□□□□□

4.

- 
- 

5. □□□□□

- [] [] [] [] [] [] qrels
- [] [] [] [] [] []
- [] [] [] [] [] [] SON [] Markdown []

4.2 〇〇〇〇〇〇

- **NDCG@k**: Normalized Discounted Cumulative Gain
- **MAP**: Mean Average Precision
- **Recall@k**: 
- **Precision@k**: 
- **MRR**: Mean Reciprocal Rank
- **Recall Cap**: 

utils.py

4.3 〇〇〇〇〇〇

JSON ☐☐☒

```
{
  "model_name": {
    "reranker_name": {
      "split": {
        "ndcg_at_10": 0.5,
        "recall_at_10": 0.6,
        ...
      }
    }
  }
}
```

Markdown ☐☐☐

[illegible]

5. 图

FlagEmbedding 图

- 1. 图
- 2. 图
- 3. 图top-k图
- 4. 图
- 5. 图

06 key algorithms

FlagEmbedding 图

图

Diagram 8

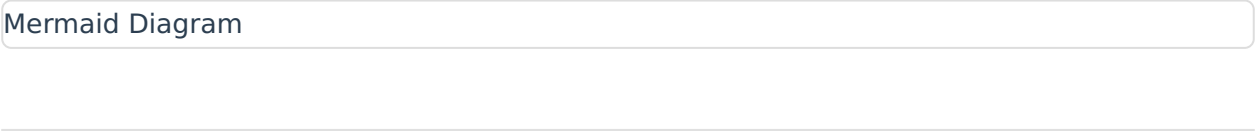
Mermaid Diagram

模型实现

名称	操作	实现	输入	输出	备注
CLS Pooling	Pooling	last_hidden_state[:,0]	Embed	Embed	Embed
Mean Pooling	Pooling	Embed	Embed	Embed	Embed
Last Token	Pooling	Embedtoken	Embed	DecoderEmbed	Embed
BatchEmbed	Embed	batchEmbed	Embed	batchEmbed	Embed
GPUEmbed	Embed	GPUEmbed	Embed	Embed	Embed
KLEmbed	Embed	teacher_scores	Embed	Embed	Embed
M3Embed	Embed	Embed	Embed	M3Embed	Embed
Dense	Embed	poolingEmbed	Embed	Embed	Embed
Sparse	Embed	Embed	Embed	Embed	Embed
CoBERT	Embed	tokenEmbed	Embed	Embed	Embed

BGE-M3 模型实现

Diagram 9



FlagEmbedding 模型实现, MRL CoBERT 模型实现

1. 模型实现

1.1 模型实现

FlagEmbedding 模型实现 Transformer 模型实现 BERT RoBERTa 模型实现

模型实现

- Tokenizer token IDs
- token IDs Embedding
- last hidden state

4. [] pooling []

5. []

1.2 Pooling []

FlagEmbedding [] pooling [] Transformer []

1.2.1 CLS Pooling

CLS pooling [] <[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]> token []

[]

- /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/modeling.py:94-95
- /workspace/FlagEmbedding/inference/embedder/encoder_only/base.py:301-302

[]

```
if self.sentence_pooling_method == "cls":
    return last_hidden_state[:, 0]
```

[]

- []
- []
- [] <[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]> token []

1.2.2 Mean Pooling

Mean pooling [] token []

[]

- /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/modeling.py:96-101
- /workspace/FlagEmbedding/inference/embedder/encoder_only/base.py:303-306

[]

```
elif self.sentence_pooling_method == "mean":
    s = torch.sum(
        last_hidden_state * attention_mask.unsqueeze(-1).float(), dim=1
    )
    d = attention_mask.sum(dim=1, keepdim=True).float()
    return s / d
```

[]

- [] token []
- []
- [] attention mask [] padding tokens

1.2.3 Last Token Pooling

Last Token pooling [] token [] Decoder-Only []

[]

- /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/modeling.py:102-113

[]

```
elif self.sentence_pooling_method == "last_token":
    left_padding = attention_mask[:, -1].sum() == attention_mask.shape[0]
    if left_padding:
        return last_hidden_state[:, -1]
    else:
        sequence_lengths = attention_mask.sum(dim=1) - 1
        batch_size = last_hidden_state.shape[0]
        return last_hidden_state[
            torch.arange(batch_size, device=last_hidden_state.device),
            sequence_lengths,
        ]
```

[]

- []

- []

- []

1.3 []

[]

[]

- /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/modeling.py:104-105

- /workspace/FlagEmbedding/inference/embedder/encoder_only/base.py:263-264

[]

```
if self.normalize_embeddings:
    all_p_reps = torch.nn.functional.normalize(all_p_reps, dim=-1)
```

[]

- []

- []

- []

[]

FlagEmbedding [] MRL[]

[] /workspace/FlagEmbedding/inference/embedder/encoder_only/base.py:262

```
embeddings = self._truncate_embeddings(embeddings)
```

2. 训练模型

2.1 数据加载

训练数据加载器

2.2 模型定义

FlagEmbedding 模型定义

训练过程

- /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/modeling.py:56
- /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/modeling.py:171-181

训练结果

```
self.cross_entropy = torch.nn.CrossEntropyLoss(reduction='mean')

def compute_loss(self, scores, target):
    return self.cross_entropy(scores, target)
```

2.3 模型评估

模型评估器

训练过程

- /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/modeling.py:143-169

训练结果

```
def compute_score(self, q_reps, p_reps):
    scores = self._compute_similarity(q_reps, p_reps) / self.temperature
    scores = scores.view(q_reps.size(0), -1)
    return scores

def _compute_similarity(self, q_reps, p_reps):
    if len(p_reps.size()) == 2:
        return torch.matmul(q_reps, p_reps.transpose(0, 1))
    return torch.matmul(q_reps, p_reps.transpose(-2, -1))
```

模型评估结果

- 模型评估结果

- 0.01 1.0

3. In-batch Negatives

FlagEmbedding

3.1 In-batch Negatives

batch

- /workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py:171-201

```
def _compute_in_batch_neg_loss(self, q_reps, p_reps, teacher_targets=None, compute_score_func=None, **kwargs):
    group_size = p_reps.size(0) // q_reps.size(0)

    if compute_score_func is None:
        scores = self.compute_score(q_reps, p_reps)
    else:
        scores = compute_score_func(q_reps, p_reps, **kwargs)

    if teacher_targets is not None:
        #
        ...
    else:
        idxs = torch.arange(q_reps.size(0), device=q_reps.device, dtype=torch.long)
        targets = idxs * group_size
        loss = self.compute_loss(scores, targets)

    return scores, loss
```

1. batch batch
- 2.
- 3.

-
-
-

3.2 Cross-device Negatives

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100

100000

- /workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py:203-241

100000

```
def _compute_cross_device_neg_loss(self, q_reps, p_reps, teacher_targets=None, compute_score_func=None):
    group_size = p_reps.size(0) // q_reps.size(0)

    cross_q_reps = self._dist_gather_tensor(q_reps)
    cross_p_reps = self._dist_gather_tensor(p_reps)

    if compute_score_func is None:
        cross_scores = self.compute_score(cross_q_reps, cross_p_reps)
    else:
        cross_scores = compute_score_func(cross_q_reps, cross_p_reps, **kwargs)

    #
    ...

    return cross_scores, loss
```

100000000

```
def _dist_gather_tensor(self, t: Optional[torch.Tensor]):
    if t is None:
        return None
    t = t.contiguous()

    all_tensors = [torch.empty_like(t) for _ in range(self.world_size)]
    dist.all_gather(all_tensors, t)

    all_tensors[self.process_rank] = t
    all_tensors = torch.cat(all_tensors, dim=0)

    return all_tensors
```

1000

- 1000000000

- 100000 = 1000 × batch_size

- 100000 PyTorch Distributed 100000

3.3

1000 no_in_batch_neg_flag=True 1000000000

100000

- /workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py:149-169

```
def _compute_no_in_batch_neg_loss(self, q_reps, p_reps, teacher_targets=None, compute_score_func=None,
    group_size = p_reps.size(0) // q_reps.size(0)

    local_scores = self.compute_local_score(q_reps, p_reps, compute_score_func, **kwargs)

    if teacher_targets is not None:
        loss = self.distill_loss(self.kd_loss_type, teacher_targets, local_scores, group_size=group_size)
        if self.kd_loss_type == "kl_div":
            local_targets = torch.zeros(local_scores.size(0), device=local_scores.device, dtype=torch.float)
            loss += self.compute_loss(local_scores, local_targets)
    else:
        local_targets = torch.zeros(local_scores.size(0), device=local_scores.device, dtype=torch.long)
        loss = self.compute_loss(local_scores, local_targets)

    return local_scores, loss
```

- /workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py:319-324

```
if kd_loss_type == 'kl_div':
    return - torch.mean(
        torch.sum(torch.log_softmax(student_scores, dim=-1) * teacher_targets, dim=-1)
    )
```

1. `teacher_scores`
2. `softmax` `teacher_targets`
3. `student_scores`
4. `KL`
5.

```

if teacher_scores is not None:
    teacher_scores = torch.tensor(teacher_scores, device=device)
    teacher_scores = teacher_scores.view(batch_size, -1).detach()
    teacher_targets = F.softmax(teacher_scores, dim=-1)
else:
    teacher_targets = None

```

4.2 M3 KD

M3

- /workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py:325-340

```

elif kd_loss_type == 'm3_kd_loss':
    labels = torch.arange(student_scores.size(0), device=student_scores.device, dtype=torch.long)
    labels = labels * group_size

    loss = 0
    mask = torch.zeros_like(student_scores)
    for i in range(group_size):
        temp_target = labels + i
        temp_scores = student_scores + mask
        temp_loss = F.cross_entropy(temp_scores, temp_target, reduction="none")
        loss += torch.mean(teacher_targets[:, i] * temp_loss)
        mask = torch.scatter(mask, dim=-1, index=temp_target.unsqueeze(-1),
                             value=torch.finfo(student_scores.dtype).min)

    return loss

```

-

-

- mask

5. MRL Matryoshka Representation Learning

Matryoshka Representation Learning

5.1

MRL k

5.2 实现

实现

- /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/modeling.py:91-102
- /workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py:264-295

实现 **MRL** 实现

```
def encode(self, features):
    # ...
    if self.use_mrl:
        p_reps_list = []
        ori_dim = all_p_reps.size(-1)
        for dim in self.mrl_dims:
            if dim > ori_dim:
                logger.warning(f"MRL dim {dim} is larger than original dimension {ori_dim}, using orig")
                dim = min(dim, ori_dim)
            dim_p_reps = all_p_reps[:, :dim]
            if self.normalize_embeddings:
                dim_p_reps = torch.nn.functional.normalize(dim_p_reps, dim=-1)
            p_reps_list.append(dim_p_reps.contiguous())
        return p_reps_list
    else:
        if self.normalize_embeddings:
            all_p_reps = torch.nn.functional.normalize(all_p_reps, dim=-1)
        return all_p_reps.contiguous()
```

实现 **MRL** 实现

```
if self.use_mrl:
    all_loss = torch.tensor(0.0, device=device)
    for dim_q_reps, dim_p_reps in zip(q_reps, p_reps):
        _, mrl_loss = compute_loss_func(dim_q_reps, dim_p_reps, teacher_targets=teacher_targets)
        all_loss += mrl_loss
    loss = all_loss / len(self.mrl_dims)
else:
    scores, loss = compute_loss_func(q_reps, p_reps, teacher_targets=teacher_targets)
```

MRL 实现

-
-
-
-

6. ColBERT 实现M3 实现

BGE-M3 与 FlagEmbedding 实现 ColBERT 实现

6.1 初始化

M3 初始化 encoder 初始化

- `sparse_linear`: 初始化
- `colbert_linear`: 初始化 ColBERT 初始化

初始化

- `/workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:70-72`

初始化

```
if not self.unified_finetuning:
    self.model = base_model['model']
    self.colbert_linear = None
    self.sparse_linear = None
else:
    self.model = base_model['model']
    self.colbert_linear = base_model['colbert_linear']
    self.sparse_linear = base_model['sparse_linear']
```

6.2 初始化Dense

初始化 pooling 初始化

初始化

- `/workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:81-113`

```
def _dense_embedding(self, last_hidden_state, attention_mask):
    if self.sentence_pooling_method == "cls":
        return last_hidden_state[:, 0]
    elif self.sentence_pooling_method == "mean":
        s = torch.sum(
            last_hidden_state * attention_mask.unsqueeze(-1).float(), dim=1
        )
        d = attention_mask.sum(dim=1, keepdim=True).float()
        return s / d
    # ...
```

6.3 初始化Sparse

初始化 token

初始化

- `/workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:116-157`

初始化

```

def _sparse_embedding(self, hidden_state, input_ids, return_embedding: bool = True):
    token_weights = torch.relu(self.sparse_linear(hidden_state))
    if not return_embedding:
        return token_weights

    if self.training:
        sparse_embedding = torch.zeros(
            input_ids.size(0), input_ids.size(1), self.vocab_size,
            dtype=token_weights.dtype,
            device=token_weights.device
        )
        sparse_embedding = torch.scatter(sparse_embedding, dim=-1, index=input_ids.unsqueeze(-1), src=token_weights)
        sparse_embedding = torch.max(sparse_embedding, dim=1).values
    else:
        sparse_embedding = torch.zeros(
            input_ids.size(0), self.vocab_size,
            dtype=token_weights.dtype,
            device=token_weights.device
        )
        sparse_embedding = sparse_embedding.scatter_reduce(
            dim=-1, index=input_ids, src=token_weights.squeeze(-1), reduce="amax"
        )

    unused_tokens = [
        self.tokenizer.cls_token_id, self.tokenizer.eos_token_id,
        self.tokenizer.pad_token_id, self.tokenizer.unk_token_id
    ]
    sparse_embedding[:, unused_tokens] *= 0.
    return sparse_embedding

```

□□□□□□

- □□□□□□

- □□□□

- □□□□□□□□

- □□□ BM25 □□□□□□□□

□□□□□□

□□□□□□□□□□ token □□□□□□

```

def _process_token_weights(token_weights: np.ndarray, input_ids: list):
    result = defaultdict(int)
    unused_tokens = set()
    for _token in ['cls_token', 'eos_token', 'pad_token', 'unk_token']:
        if _token in self.tokenizer.special_tokens_map:
            _token_id = self.tokenizer.convert_tokens_to_ids(self.tokenizer.special_tokens_map[_token])
            unused_tokens.add(_token_id)
    for w, idx in zip(token_weights, input_ids):
        if idx not in unused_tokens and w > 0:
            idx = str(idx)
            if w > result[idx]:
                result[idx] = w
    return result

```

6.4 CoBERT

CoBERT token <[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]>
token

- /workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:159-171

```
def _colbert_embedding(self, last_hidden_state, mask):
    colbert_vecs = self.colbert_linear(last_hidden_state[:, 1:])
    colbert_vecs = colbert_vecs * mask[:, 1:][:, :, None].float()
    return colbert_vecs
```

CoBERT

```
def compute_colbert_score(self, q_reps, p_reps, q_mask: torch.Tensor = None):
    token_scores = torch.einsum('qin,pjn->qipj', q_reps, p_reps)
    scores, _ = token_scores.max(-1)
    scores = scores.sum(1) / q_mask[:, 1:].sum(-1, keepdim=True)
    scores = scores / self.temperature
    return scores
```

CoBERT

- token
-
-

6.5

M3

- /workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:374-476


```
def forward(self, queries, passages, teacher_scores=None, no_in_batch_neg_flag=False):
    q_dense_vecs, q_sparse_vecs, q_colbert_vecs = self.encode(queries)
    p_dense_vecs, p_sparse_vecs, p_colbert_vecs = self.encode(passages)

    if self.training:
        # Dense
        dense_scores, loss = compute_loss_func(
            q_dense_vecs, p_dense_vecs, teacher_targets=teacher_targets,
            compute_score_func=self.compute_dense_score
        )

    if self.unified_finetuning:
        # Sparse
        sparse_scores, sparse_loss = compute_loss_func(
            q_sparse_vecs, p_sparse_vecs, teacher_targets=teacher_targets,
            compute_score_func=self.compute_sparse_score
        )

        # ColBERT
        colbert_scores, colbert_loss = compute_loss_func(
            q_colbert_vecs, p_colbert_vecs, teacher_targets=teacher_targets,
            compute_score_func=self.compute_colbert_score,
            q_mask=self._get_queries_attention_mask(queries)
        )

        # Ensemble
        ensemble_scores, ensemble_loss = compute_loss_func(
            q_dense_vecs, p_dense_vecs, teacher_targets=teacher_targets,
            compute_score_func=self.ensemble_score,
            dense_scores=dense_scores,
            sparse_scores=sparse_scores,
            colbert_scores=colbert_scores
        )

    loss = (loss + ensemble_loss + 0.1 * sparse_loss + colbert_loss) / 4

    # Self-distillation
    if self.use_self_distill and self.step > self.self_distill_start_step:
        self_teacher_targets = torch.softmax(ensemble_scores.detach(), dim=-1)
        # ...

    return EmbedderOutput(loss=loss)
```

6.6 □□□□

[illegible]

□□□□□

- /workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:173-189
- /workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:239-257

□□□□□

□ □ □ □ □ □ □

7. 00

類別	項目	說明
其他	游泳池	pooling 費
其他	圖書館	
其他	圖書館	借閱/借閱費
其他	圖書館	KL 圖書館 M3 KD
MRL	圖書館	圖書館
CoIBERT & 其他	圖書館	M3 圖書館

FlagEmbedding

07 research projects

FlagEmbedding 项目合集

项目合集

Diagram 10

Mermaid Diagram

项目合集

项目	语言	数据集	模型	效果
BGE-M3	Python	多语言+多模态+多任务	多语言	SOTA 性能
LM-Cocktail	Python	多语言混合	多语言	多语言
LLM-Embedder	Python	6个数据集	LLM模型	多语言
Activation-Beacon	Python	多语言	400K模型	多语言
BGE-VL	Python	MegaPairs数据集	图-图	多语言
BGE-Code	Python	多语言	多语言	CoIR SOTA

项目合集

Diagram 11

Mermaid Diagram

项目

- 1. BGE-M3项目

2. [LM-Cocktail](#)
3. [LLM-Embedder](#)
4. [Activation-Beacon](#)
5. [BGE-VL](#)
6. [BGE-Code](#)
7. [BGE-M3](#)

BGE-M3

BGE-M3 是 FlagEmbedding 推出的 "M3" 系列模型，具有 **Multi-Functionality**、**Multi-Linguality** 和 **Multi-Granularity** 三大特性。

- [BGE_M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation](#)
- [BGE-M3](#)
- `/workspace/FlagEmbedding/inference/embedder/encoder_only/m3.py`
- `/workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/`
- `/workspace/research/BGE_M3/`

1. Multi-Functionality

BGE-M3 支持多种任务，包括：

1. **Dense Retrieval**

`/workspace/research/BGE_M3/modeling.py` 中：

```
def dense_embedding(self, hidden_state, mask):
    if self.sentence_pooling_method == 'cls':
        return hidden_state[:, 0]
    elif self.sentence_pooling_method == 'mean':
        s = torch.sum(hidden_state * mask.unsqueeze(-1).float(), dim=1)
        d = mask.sum(axis=1, keepdim=True).float()
        return s / d
```

📄📄📄📄 /workspace/FlagEmbedding/inference/embedder/encoder_only/m3.py 📄📄

- 📄📄 cls 📄 mean 📄📄📄📄📄
- 📄📄📄📄📄📄📄📄📄

📄2📄📄📄📄📄 Sparse Retrieval / Lexical Matching📄

📄📄📄 BM25 📄📄📄📄📄📄 token 📄📄📄📄📄📄

📄📄📄📄 /workspace/research/BGE_M3/modeling.py 📄📄

```
def sparse_embedding(self, hidden_state, input_ids, return_embedding: bool = True):
    token_weights = torch.relu(self.sparse_linear(hidden_state))
    if not return_embedding: return token_weights

    if self.training:
        sparse_embedding = torch.zeros(
            input_ids.size(0), input_ids.size(1), self.vocab_size,
            dtype=token_weights.dtype,
            device=token_weights.device
        )
        sparse_embedding = torch.scatter(sparse_embedding, dim=-1, index=input_ids.unsqueeze(-1), src=token_weights)
        sparse_embedding = torch.max(sparse_embedding, dim=1).values
    else:
        # 📄scatter_reduce
        sparse_embedding = torch.zeros(
            input_ids.size(0), self.vocab_size,
            dtype=token_weights.dtype,
            device=token_weights.device
        )
        sparse_embedding = sparse_embedding.scatter_reduce(
            dim=-1, index=input_ids, src=token_weights.squeeze(-1), reduce="amax"
        )

    unused_tokens = [self.tokenizer.cls_token_id, self.tokenizer.eos_token_id,
                     self.tokenizer.pad_token_id, self.tokenizer.unk_token_id]
    sparse_embedding[:, unused_tokens] *= 0.
    return sparse_embedding
```

📄📄📄📄📄📄

- 📄📄 sparse_linear 📄📄📄📄📄 token 📄📄
- 📄📄 ReLU 📄📄📄📄
- 📄📄 token📄CLS📄EOS📄PAD📄UNK📄📄📄 0
- 📄📄📄 scatter_reduce 📄📄📄

📄3📄📄📄📄📄 Multi-Vector Retrieval / ColBERT📄

📄📄 token 📄📄📄📄📄📄📄📄📄📄📄📄

📄📄📄📄 /workspace/research/BGE_M3/modeling.py 📄📄

```
def colbert_embedding(self, last_hidden_state, mask):
    colbert_vecs = self.colbert_linear(last_hidden_state[:, 1:]) # [BOS_never_used_51bce0c785ca2f68
    colbert_vecs = colbert_vecs * mask[:, 1:][:, :, None].float()
    return colbert_vecs
```

📄 /workspace/research/BGE_M3/modeling.py 11

```
def colbert_score(self, q_reps, p_reps, q_mask: torch.Tensor):
    token_scores = torch.einsum('qin,pjn->qipj', q_reps, p_reps)
    scores, _ = token_scores.max(-1) # [token]
    scores = scores.sum(1) / q_mask[:, 1:].sum(-1, keepdim=True) # []
    scores = scores / self.temperature
    return scores
```

2. 🌐Multi-Linguality🌐

BGE-M3 📄 100 📄 XLM-RoBERTa 📄

- 📄XLM-RoBERTa-large
- 📄250,000+
- 📄

3. 📄Multi-Granularity📄

📄 8192 tokens 📄

📄 /workspace/research/BGE_M3/modeling.py 11

```
def compute_sub_batch_size(self, features):
    mapping = [(6000, 1), (5000, 2), (4000, 3), (3000, 3), (2000, 5), (1000, 9), (512, 16), (0, 32)]
    cur_l = features['input_ids'].size(-1)
    for l, b in mapping:
        if cur_l >= l:
            return b
```

📄

📄

BGE-M3 📄

📄 /workspace/research/BGE_M3/modeling.py 11

```
def load_model(self, model_name, colbert_dim: int = -1):
    self.model = AutoModel.from_pretrained(model_name)
    self.tokenizer = AutoTokenizer.from_pretrained(model_name)

    # □
    self.colbert_linear = torch.nn.Linear(in_features=self.model.config.hidden_size,
                                           out_features=self.model.config.hidden_size if co
    self.sparse_linear = torch.nn.Linear(in_features=self.model.config.hidden_size, out_fe
```

```
../../../../workspace/research/BGE_M3/modeling.py
```

```
if self.unified_finetuning:
    # Sparse Scores
    sparse_scores = self.sparse_score(q_sparse_vecs, p_sparse_vecs)
    sparse_loss = self.compute_loss(sparse_scores, targets)

    # Colbert Scores
    colbert_scores = self.colbert_score(q_colbert_vecs, p_colbert_vecs, q_mask=query['attention_mask'], p_mask=passage['attention_mask'])
    colbert_loss = self.compute_loss(colbert_scores, targets)

    # Ensemble Loss
    ensemble_loss = self.compute_loss(dense_scores + 0.3 * sparse_scores + colbert_scores, targets)

    # Final Loss
    loss = (loss + ensemble_loss + 0.1 * sparse_loss + colbert_loss) / 4
```

Self-Knowledge Distillation

BGE-M3

```
../../../../workspace/research/BGE_M3/modeling.py
```

```

if self.use_self_distill and self.step > self.self_distill_start_step and self.unified_fin
    ensemble_scores = dense_scores + 0.3 * sparse_scores + colbert_scores
    teacher_targets = torch.softmax(ensemble_scores.detach(), dim=-1)

    ensemble_distill_dense_loss = - torch.mean(
        torch.sum(torch.log_softmax(dense_scores, dim=-1) * teacher_targets, dim=-1))
    ensemble_distill_sparse_loss = - torch.mean(
        torch.sum(torch.log_softmax(sparse_scores, dim=-1) * teacher_targets, dim=-1))
    ensemble_distill_colbert_loss = - torch.mean(
        torch.sum(torch.log_softmax(colbert_scores, dim=-1) * teacher_targets, dim=-1))

    loss += (ensemble_distill_dense_loss + 0.1 * ensemble_distill_sparse_loss + ensemble_d
    loss = loss / 2

```

□ □ □ □ □ □

M3Embedder `/workspace/FlagEmbedding/inference/embedder/encoder_only/m3.py`

--	--	--	--	--

1. 
2. 

3. ColBERT

4.

```
def encode_single_device(self, sentences, batch_size=256, max_length=8192,
                        return_dense=True, return_sparse=False, return_colbert_vecs=False):
    # ...
    # padding
    length_sorted_idx = np.argsort([-len(x['input_ids']) for x in all_inputs])
    # ...
    # batch size 100M
    while flag is False:
        try:
            # ...
        except RuntimeError as e:
            batch_size = batch_size * 3 // 4
```

```
def compute_score_single_device(self, sentence_pairs, batch_size=256,
                                max_query_length=512, max_passage_length=512,
                                weights_for_different_modes=None):
    # ...
    #
    if weights_for_different_modes is None:
        weights_for_different_modes = [1., 1., 1.]
        weight_sum = 3
    # ...
    all_scores['sparse+dense'] = (sparse_scores * w[1] + dense_scores * w[0]) / (w[1] + w[0])
    all_scores['colbert+sparse+dense'] = (colbert_scores * w[2] + sparse_scores * w[1] + dense_scores * w[0]) / (w[2] + w[1] + w[0])
```

LM-Cocktail

LM-Cocktail

1.

2.

- LM-Cocktail: Resilient Tuning of Language Models via Model Merging
- `/workspace/research/LM-Cocktail/`

$$\text{Merged} = w_1 * M_1 + w_2 * M_2 + \dots + w_n * M_n$$

1.

```
python /workspace/research/LM_Cocktail/LM_Cocktail/cocktail.py
```

```
def mix_models(model_names_or_paths: List[str],
               model_type: str,
               weights: List[float],
               output_path: str=None):
    """
    mix models based on given weights
    """
    assert len(model_names_or_paths) == len(weights)
    assert model_type in ['decoder', 'encoder', 'reranker']
    assert sum(weights) - 1 <= 1e-3

    # []
    param_list = get_model_param_list(model_names_or_paths, model_type=model_type)
    # []
    new_param = merge_param(param_list, weights=weights)

    # []
    model = load_model(model_names_or_paths[0], model_type=model_type)
    model.load_state_dict(new_param)

    # []
    if output_path is not None:
        model.save_pretrained(output_path)
        tokenizer = AutoTokenizer.from_pretrained(model_names_or_paths[0], trust_remote_code=True)
        tokenizer.save_pretrained(output_path)
    return model
```

2. 問題

```
../../../../workspace/research/LM_Cocktail/LM_Cocktail/cocktail.py
```

```

def mix_models_with_data(model_names_or_paths: List[str],
                        model_type: str,
                        example_data: List[Dict],
                        temperature: float=5.0,
                        batch_size:int=2,
                        max_input_length:int=2048,
                        neg_number: int=7,
                        output_path: str=None):

    """
    mix model based on given a few examples
    """

    # 1.
    weights = compute_weights(model, tokenizer=tokenizer, param_list=param_list,
                              model_type=model_type, example_data=example_data,
                              temperature=temperature, neg_number=neg_number,
                              batch_size=batch_size, max_input_length=max_input_length)

    # 2.
    new_param = merge_param(param_list, weights=weights)
    model.load_state_dict(new_param)

    return model

```

3.

/workspace/research/LM_Cocktail/LM_Cocktail/cocktail.py

```

def mix_models_by_layers(model_names_or_paths: List[str],
                        model_type: str,
                        weights: List[float],
                        output_path: str=None):

    """
    mix models based on given weights, and load them layer by layer
    """

    #
    param_dirs, temp_dir = get_model_param_dirs(model_names_or_paths, model_type=model_type)
    temp_file_path = merge_param_by_layer(param_dirs, weights=weights)

    # accelerate
    with init_empty_weights():
        meta_model = AutoModelForCausalLM.from_pretrained(...) #

    device_map = {name: "cpu" for name, _ in meta_model.named_modules()}
    model = load_checkpoint_and_dispatch(meta_model, checkpoint=temp_file_path, device_map=device_map)

    #
    os.remove(temp_file_path)
    shutil.rmtree(temp_dir)

    return model

```

--	--	--	--

1. 
2. 
3. 

--	--	--	--

- 
- 
- 

LLM-Embedder LLM

□□□□

LLM-Embedder

- [Retrieval Augmented Generation \(RAG\)](#)
- `./workspace/research/llm_embedder/`

--	--	--	--	--	--	--

LLM-Embedder 6

Topic	Progress
QA (100)	██████████
Conversational Search (1000)	██████████
Long Conversation (100)	████████
Long-Range Language Modeling (10000)	████████████████
In-Context Learning (10000)	████████
Tool Learning (1000)	████████

□ □ □ □ □ □

[illegible]

□ □ □ □

- `../../../../workspace/research/llm_embedder/evaluation/`
- `../../../../BM25../../../../`

□ □ □ □

- [Soaring from 4K to 400K: Extending LLM's Context with Activation Beacon](#)
- `/workspace/research/Long_LLM/activation beacon/`

1111

1. 
2. 
3. 

1111

- 
- 
-  LLM  Llama  Mistral  Qwen 
- 

□□□□□

- Mistral
- Llama-3
- Qwen-2
-

--	--	--	--

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "namespace-Pt/beacon-qwen-2-7b-instruct"
tokenizer = AutoTokenizer.from_pretrained(model_id, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(model_id, trust_remote_code=True)

# []

model.memory.reset()

# []

messages = [{"role": "user", "content": very_long_text}]
inputs = tokenizer.apply_chat_template(messages, return_tensors="pt")
outputs = model.generate(**inputs)
```

BGE-VL

□□□□

BGE-VL □ BGE □□□□□□□□□□□□□□□□□□□□

- [📄📄 MegaPairs: Massive Data Synthesis For Universal Multimodal Retrieval](#)
- [📄📄📄📄 /workspace/research/BGE_VL/](#)

□ □ □ □

模型	训练数据	性能
BGE-VL-base	CLIP 预训练	~70.0
BGE-VL-large	CLIP 预训练	~70.0
BGE-VL-MLLM-S1	LLaVA-NeXT	~75.0 MegaPairs 数据集 SOTA
BGE-VL-MLLM-S2	LLaVA-NeXT	S1 + MMEB 数据集

MegaPairs

MegaPairs 2600

- KNN + →
-
-

- 1.
- 2.
3. + →
- 4.

BGE-Code

BGE-CodeCodeR

- [BAAI/bge-code-v1](#)
- `/workspace/research/BGE_Coder/`

1. SOTA
- 2.
3. 20+
4. 4096 tokens

目录

模块	子模块
基础	基础模型
应用	应用模型
工具	工具模型
数据	数据模型
部署	部署模型
评估	评估模型

目录

CoIRCode Retrieval BenchmarkCodeRAG BGE-Code

目录

1. BGE-Reranker & LLM-Reranker

-
- FlagEmbedding/inference/reranker/
- research/llm_reranker/
- LLM

2. C-MTEB & MLDR

- C-MTEBMTEB
- MLDR13
- /workspace/research/C_MTEB/

3. LongLLM-QLoRA

QLoRALlama-380K

- /workspace/research/Long_LLM/longllm_qlora/

- 

4. Visualized-BGE

 token  BGE 

- 
- 

5. Matryoshka-Reranker

 Matryoshka 

6. LLARA

LLM 

7. Reinforced-IR





FlagEmbedding 

1.  BGE  v1.5  M3  Code  VL
2.  LM-Cocktail  Activation-Beacon 
3.  LLM-Embedder  LLM  Reranker 
4.  MegaPairs  MLDR  C-MTEB

